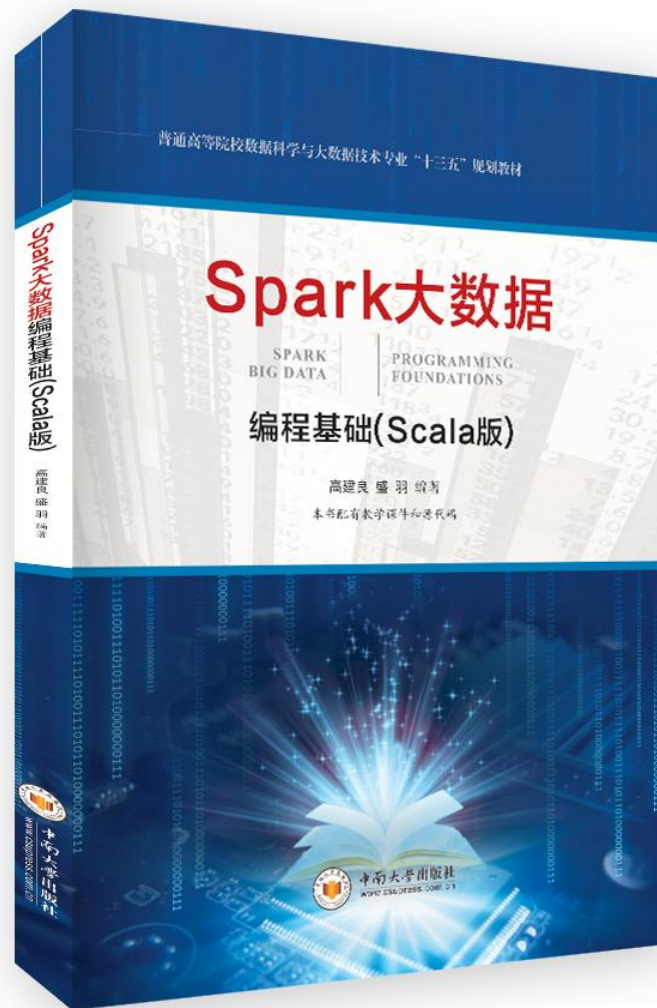
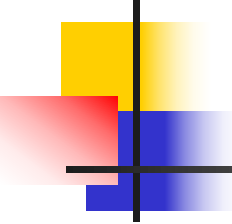




Spark大数据 编程基础 (Scala版)

<http://aibigdata.csu.edu.cn>

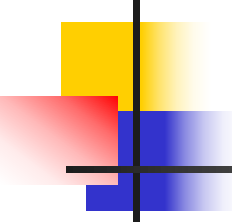




Spark大数据编程基础（Scala版）

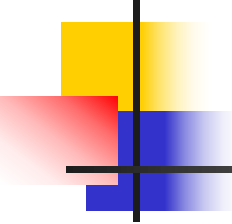
第二章 搭建Spark开发环境

工欲善其事，必先利其器。学习Spark编程，首先应搭建好Spark开发环境。本章将介绍Linux下Spark开发环境的搭建步骤。本章包括JDK配置、Spark Shell环境配置、使用IDEA开发并生成jar包，并通过spark-submit提交运行，以及使用Eclipse开发并运行Spark程序。完成Spark开发环境搭建后，即可进一步进行应用开发。



主要内容

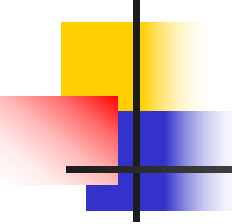
- 2.1 Spark开发环境所需软件
- 2.2 安装Spark
- 2.3 IDEA
- 2.4 Eclipse
- 2.5 本章小结



2.1 Spark开发环境所需软件

Spark可以运行在Linux或Windows系统上。由于Spark在Linux系统上运行效率更高，当前服务器大多使用Linux系统部署Spark。本书以Ubuntu 16.04为例，在Linux系统上搭建Spark开发环境。

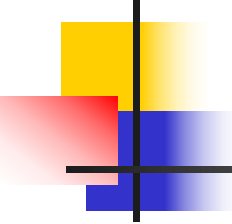
常见的Spark部署模式主要分为**Local**（本地）部署、**Standalone**（独立）部署、**YARN**部署以及**Mesos**模式。Local模式部署Spark开发环境比较简单，Spark能通过内置的单机集群调度器在本地运行。在Local模式下，所有的Spark进程运行在同一个Java虚拟机中，实际上构成一个独立、多线程版本的Spark环境。**Local**模式适合程序的原型设计、开发、调试及测试。



2.1 Spark开发环境所需软件

以部署Spark2.3.0为例。涉及的软件配置如表所示。

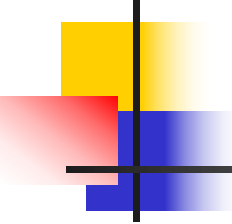
软件名	软件版本号
OS	64位Linux Ubuntu 16.04
JDK	jdk-8u171-linux-x64.tar.gz
Spark	spark-2.3.0-bin-hadoop2.7
Scala	2.11.8
IDEA	IDEA Community 2018.1.6 for Linux
Eclipse	Oxygen.3a(4.7.3a) for Linux
IDEA的Scala插件	Community 2018.1.9 for Linux
Eclipse的Scala插件	4.7.0



2.2 安装Spark

前期准备中需要下载JDK和Spark的安装包。具体版本号和官方下载地址如表所示：

软件名	版本号	下载地址
JDK	JDK8或更高版本	https://www.oracle.com/technetwork/java/javase/downloads/java-archive-javase8-2177648.html
Spark	spark-2.3.0-bin-hadoop2.7	https://archive.apache.org/dist/spark/spark-2.3.0/spark-2.3.0-bin-hadoop2.7.tgz

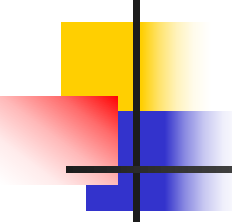


下载安装包（JDK、Spark）

在Linux系统下，可以使用两种方式下载软件安装包：

- 使用ubuntu的Firefox浏览器下载（以JDK为例）
- 使用wget命令下载（以Spark为例）

清华源：<https://mirrors.tuna.tsinghua.edu.cn/>



使用浏览器下载（以JDK为例）

使用Ubuntu自带的Firefox浏览器下载安装包。以下载JDK为例。版本号是jdk-8u171-linux-x64。

第一步，在浏览器地址栏中输入本书提供的网址“<https://www.oracle.com/technetwork/java/javase/downloads/java-archive-javase8-2177648.html>”，并回车跳转至下载页面；

注意：由于从官方网站下载旧版本JDK时，必须先使用Oracle账户登录，才能获取到一个临时的下载链接（一段时间后链接会自动失效），所以本书不提供JDK的直接下载地址。

使用浏览器下载（以JDK为例）

Java SE Development Kit 8u171

You must accept the [Oracle Binary Code License Agreement for Java SE](#) to download this software.



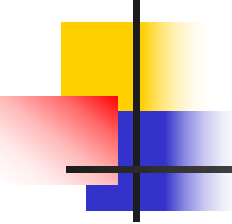
Accept License Agreement



Decline License Agreement

Product / File Description	File Size	Download
Linux ARM 32 Hard Float ABI	77.97 MB	jdk-8u171-linux-arm32-vfp-hflt.tar.gz
Linux ARM 64 Hard Float ABI	74.89 MB	jdk-8u171-linux-arm64-vfp-hflt.tar.gz
Linux x86	170.05 MB	jdk-8u171-linux-i586.rpm
Linux x86	184.88 MB	jdk-8u171-linux-i586.tar.gz
Linux x64	167.14 MB	jdk-8u171-linux-x64.rpm
Linux x64	182.05 MB	jdk-8u171-linux-x64.tar.gz
Mac OS X x64	247.84 MB	jdk-8u171-macosx-x64.dmg
Solaris SPARC 64-bit (SVR4 package)	139.83 MB	jdk-8u171-solaris-sparcv9.tar.Z
Solaris SPARC 64-bit	99.19 MB	jdk-8u171-solaris-sparcv9.tar.gz
Solaris x64 (SVR4 package)	140.6 MB	jdk-8u171-solaris-x64.tar.Z
Solaris x64	97.05 MB	jdk-8u171-solaris-x64.tar.gz
Windows x86	199.1 MB	jdk-8u171-windows-i586.exe
Windows x64	207.27 MB	jdk-8u171-windows-x64.exe

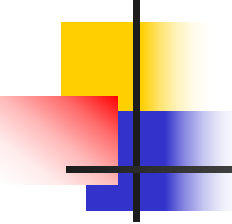
[Back to top](#)



使用浏览器下载（以JDK为例）

第二步，选中图中【Accept License Agreement】并在Download列表中找到jdk-8u171-linux-x64.tar.gz，鼠标左键点击该链接，将跳转到Oracle账户页面登录页面；

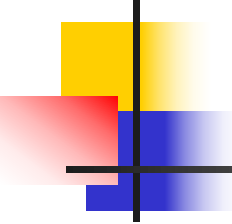
注意：本书使用的Ubuntu16.04为64位版本，对应下载Linux x64的JDK。如果使用32位的Linux系统，则下载Linux x86的JDK。



使用浏览器下载（以JDK为例）

第三步，用**Oracle**账户登录，如果没有账户请先进行注册，注册过程此处省略；

第四步，登录成功后会弹出下载框，确认即可下载。
JDK会保存在浏览器设置的下载文件夹中（**Firefox**的默认下载文件夹是“~/Downloads”）。



使用wget命令下载（以Spark为例）

以下载Spark为例，介绍通过wget命令和本书中提供的官方下载地址获取Spark安装包。

浏览器打开Spark的官方下载网站：<http://spark.apache.org/downloads.html>。在页面中设置要下载的Spark版本，以本书的Spark版本为例。

使用wget命令下载（以Spark为例）

Download Apache Spark™

1. Choose a Spark release:
2. Choose a package type:
3. Download Spark: [spark-2.3.0-bin-hadoop2.7.tgz](#)
4. Verify this release using the [2.3.0](#)

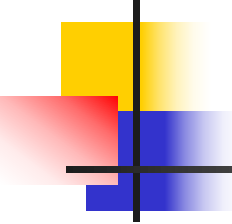
Note: Starting version 2.0, Spark is built with Scala 2.10 support.

Link with Spark

Spark artifacts are hosted in Maven Central

- Open Link in New Tab
- Open Link in New Window
- Open Link in New Private Window
- Bookmark This Link
- Save Link As...
- Save Link to Pocket
- Copy Link Location**

Spark release选择【2.3.0】， package type选择【Pre-built for Apache Hadoop 2.7 and later】。鼠标右键点击Download Spark之后的链接，然后点击菜单中的【Copy Link Location】复制下载链接。



使用wget命令下载（以Spark为例）

获取下载链接后，使用wget命令下载安装包的具体步骤为：

- 1) 打开终端。在桌面上点击鼠标右键，再点击弹出菜单中的【Open in Terminal】；
- 2) 在终端中输入命令：wget+空格，然后在终端中右键点击【Paste】复制Spark的下载链接，完整命令：

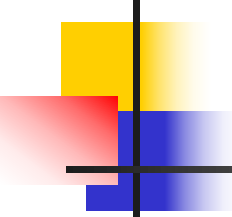
```
wget https://archive.apache.org/dist/spark/spark-2.3.0/spark-2.3.0  
-bin-hadoop2.7.tgz
```

使用wget命令下载（以Spark为例）

```
ubuntu@ubuntu: ~  
ubuntu@ubuntu:~$ wget https://archive.apache.org/dist/spark/spark-2.3.0/spark-2.3.0-bin-hadoop2.7.tgz  
--2018-11-01 09:44:56-- https://archive.apache.org/dist/spark/spark-2.3.0/spark-2.3.0-bin-hadoop2.7.tgz  
Resolving archive.apache.org (archive.apache.org)... 163.172.17.199  
Connecting to archive.apache.org (archive.apache.org)|163.172.17.199|:443... connected.  
HTTP request sent, awaiting response... 200 OK  
Length: 226128401 (216M) [application/x-gzip]  
Saving to: 'spark-2.3.0-bin-hadoop2.7.tgz'  
  
spark-2.3.0- 0%[ ] 40.00K 21.8KB/s
```

3) 回车运行，连接服务器后自动开始下载。

注意：如果不使用“-P”参数设置保存位置，下载的文件会保存在用户文件夹下（本书为“/home/ubuntu/”目录下，其中ubuntu为用户名）。



使用wget命令下载

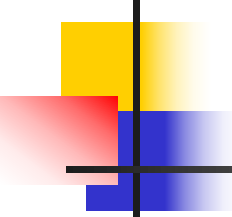
wget的常用参数使用方法（以下载Spark压缩包为例）：

1) “-O 文件名”，将下载的文件保存为自定义文件名，例如将下载的Spark压缩包自定义文件名为spark.tgz:

```
wget -O spark.tgz https://archive.apache.org/dist/spark/spark-2.3.0/spark-2.3.0-bin-hadoop2.7.tgz
```

2) “-c”，使用断点续传功能，防止下载时因网络中断导致下载失败。如果加入了-c参数后出现网络中断的情况，只需要再次运行该命令即可从之前的进度开始继续下载。

```
wget -c https://archive.apache.org/dist/spark/spark-2.3.0/spark-2.3.0-bin-hadoop2.7.tgz
```



使用wget命令下载

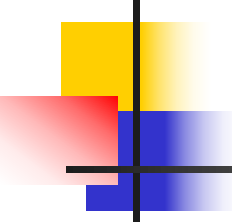
wget的常用参数使用方法（以下载Spark压缩包为例）：

3）“-P 保存路径”，将下载的文件保存到指定目录下，如“~/Downloads”目录下：

```
wget -P ~/Downloads https://archive.apache.org/dist/spark/spark-2.3.0/spark-2.3.0-bin-hadoop2.7.tgz
```

4）以上参数也可以组合使用。例如开启断点续传功能，并将下载的文件命名为“spark.tgz”并保存到用户文件夹中的Downloads文件夹下：

```
wget -P ~/Downloads -O spark.tgz -c https://archive.apache.org/dist/spark/spark-2.3.0/spark-2.3.0-bin-hadoop2.7.tgz
```



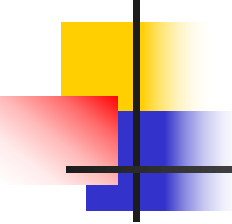
环境配置

Local模式快速部署Spark，只需解压JDK和Spark的安装包到指定目录，并在环境变量文件中配置JDK和Spark的存放路径。

(1) JDK

JDK的配置过程分为三步：

- 1) 解压安装包，安装JDK；
- 2) 配置环境变量；
- 3) 检查安装状态。



环境配置

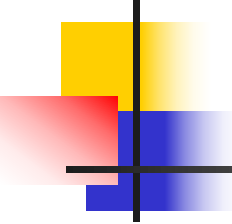
1) 解压安装包，安装**JDK**

在JDK安装包存放的目录（本书为“/home/ubuntu/Downloads/jdk-8u171-linux-x64.tar.gz”），鼠标右键点击【Open in Terminal】，打开终端，将JDK安装包解压到/usr/local目录下，解压命令：

```
sudo tar zxvf jdk-8u171-linux-x64.tar.gz -C /usr/local
```

2) 配置环境变量。终端中输入：`sudo gedit ~/.bashrc`

回车运行，在打开的**.bashrc**文档最后加入字段。



环境配置

回车运行，在打开的**.bashrc**文档最后加入字段。

```
#JAVA
```

```
export JAVA_HOME=/usr/local/jdk1.8.0_171
```

```
export JRE_HOME=${JAVA_HOME}/jre
```

```
export CLASSPATH=.:${JAVA_HOME}/lib:${JRE_HOME}/lib
```

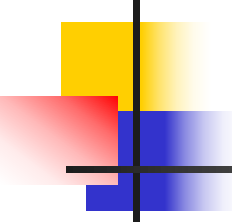
```
export PATH=$PATH:$JAVA_HOME/bin
```

注意：**JAVA_HOME**的值应与**JDK**解压后存放路径一致。

添加字段以后，点击【**Save**】保存修改。在终端中输入：

```
source ~/.bashrc
```

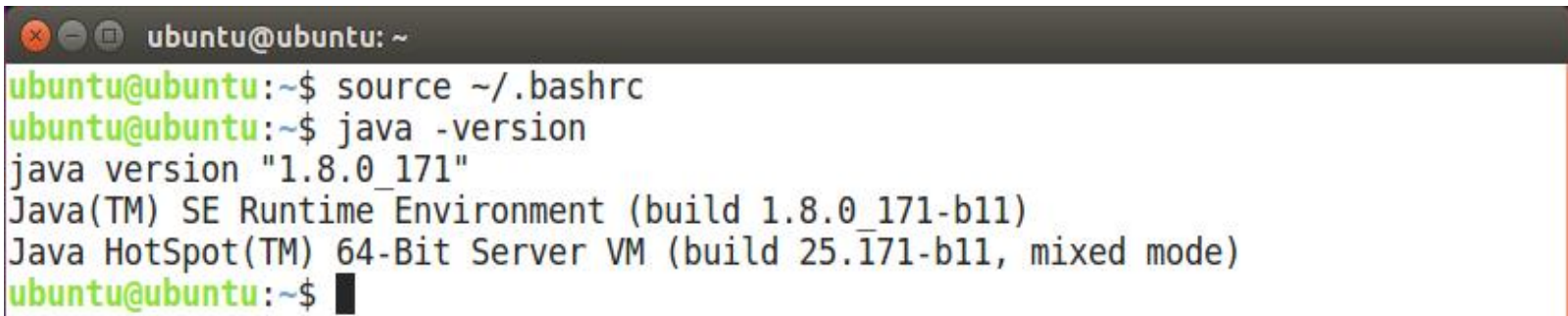
执行命令使配置生效。



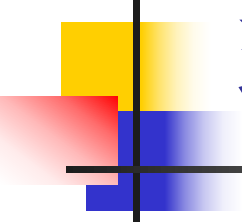
环境配置

3) 检查JDK安装状态。

终端中输入：**java -version**。终端中出现如图所示的结果，说明JDK配置成功。



```
ubuntu@ubuntu: ~  
ubuntu@ubuntu:~$ source ~/.bashrc  
ubuntu@ubuntu:~$ java -version  
java version "1.8.0_171"  
Java(TM) SE Runtime Environment (build 1.8.0_171-b11)  
Java HotSpot(TM) 64-Bit Server VM (build 25.171-b11, mixed mode)  
ubuntu@ubuntu:~$
```

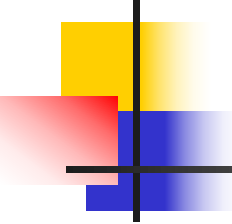


环境配置

(2) Spark

Spark的配置过程分为三步：

- 1) 解压安装包；
- 2) 配置环境变量；
- 3) 检查安装状态。



环境配置

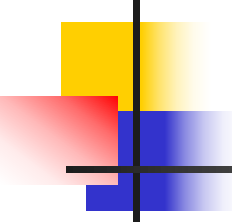
1) 解压Spark安装包

在Spark安装包存放的目录（本书为“/home/ubuntu/Downloads/ spark-2.3.0-bin-hadoop2.7.tgz”），鼠标右键点击【Open in Terminal】，打开终端，将Spark压缩包解压到“/usr/local”目录下，解压命令：

```
sudo tar zxvf spark-2.3.0-bin-hadoop2.7.tgz -C /usr/local
```

2) 配置环境变量

终端中输入：`sudo gedit ~/.bashrc`。回车运行。



环境配置

在.bashrc文档最后加入字段：

```
#SPARK
```

```
export SPARK_HOME=/usr/local/spark-2.3.0-bin-hadoop2.7
```

```
export PATH=$PATH:$SPARK_HOME/bin
```

注意：SPARK_HOME的值应与Spark解压后存放路径一致。

添加字段以后，保存退出，然后在终端中输入命令使配置生效：

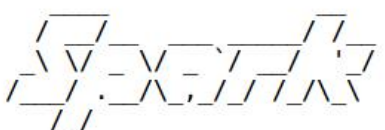
```
source ~/.bashrc。
```

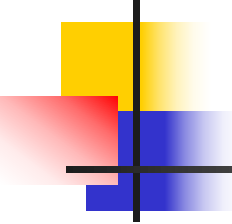
环境配置

3) 检查Spark安装状态

在终端中输入：**spark-shell**。回车运行，终端中出现如图所示信息，说明Spark配置成功。

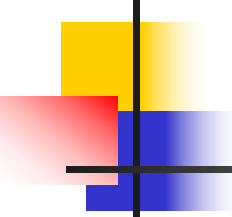
注意：IDEA中
Scala的版本应
和终端中Scala
的版本一致。

```
csubigdata@ubuntu: ~  
k address: 127.0.1.1; using 192.168.241.131 instead (on interface ens33)  
2018-05-15 05:16:05 WARN  Utils:66 - Set SPARK_LOCAL_IP if you need to bind to a  
nother address  
2018-05-15 05:16:07 WARN  NativeCodeLoader:62 - Unable to load native-hadoop lib  
rary for your platform... using builtin-java classes where applicable  
Setting default log level to "WARN".  
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLeve  
l(newLevel).  
Spark context Web UI available at http://192.168.241.131:4040  
Spark context available as sc (master = local[*], app id = local-1526386591536  
).  
Spark session available as 'spark'.  
Welcome to  
  
           version 2.3.0  
  
Using Scala version 2.11.8 (Java HotSpot(TM) 64-Bit Server VM, Java 1.8.0_171)  
Type in expressions to have them evaluated.  
Type :help for more information.  
  
scala>
```



2.2.1 Spark-shell下的实例

在Ubuntu中，打开终端，输入`spark-shell`，启动spark-shell。spark-shell启动过程中，初始化了一个SparkContext对象sc。在spark-shell中，输入代码命令后，执行产生的结果的值与类型将显示在终端中。



2.2.1 Spark-shell下的实例

在shell中，运行第一个Spark实例，步骤为：

1) 创建一个RDD。从本地目录传入文件，记为rdd1。输入：

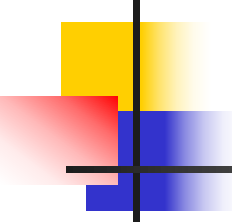
```
val rdd1 = sc.textFile("file:///usr/local/sparkwc")
```

注意：file:///表示系统根目录。sparkwc文件当前在上述路径中不存在。回车运行，结果显示为：

```
rdd1: org.apache.spark.rdd.RDD[String] = file:///usr/local/sparkwc  
MapPartitions
```

```
Rdd[1] at textFile at <console>:24
```

运行结果未报错。因为此操作属于Transformation操作，只记录逻辑关系。



2.2.1 Spark-shell下的实例

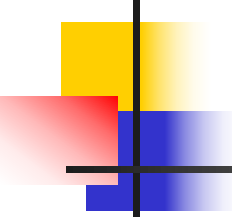
在shell中，运行第一个Spark实例，步骤为：

2) 将rdd1中的文本按"\t"进行切割分词，生成rdd2。输入：

```
val rdd2 = rdd1.flatMap(_.split("\t"))
```

回车运行，结果显示为：

```
rdd2: org.apache.spark.rdd.RDD[String] = MapPartitionsRDD[2] at  
flatMap at <console>:25
```



2.2.1 Spark-shell下的实例

在shell中，运行第一个Spark实例，步骤为：

3) 在“usr/local”目录下创建一个名为sparkwc的文件。新终端中输入：

```
sudo gedit /usr/local/sparkwc
```

4) 回车运行，在sparkwc文件中添加单词，单词之间用Tab键分开。

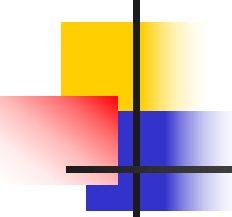
```
hadoop spark
```

```
scala    hello
```

```
hello    spark
```

```
Java     python
```

```
R        spark
```



2.2.1 Spark-shell下的实例

在shell中，运行第一个Spark实例，步骤为：

5) 将rdd2中的单词转换为形如（单词，1）的元组，生成rdd3。输入：

```
val rdd3 = rdd2.map( (_,1))
```

回车运行，结果显示为：

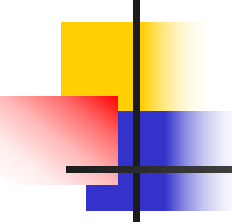
```
rdd3: org.apache.spark.rdd.RDD[(String, Int)] = MapPartitionsRDD[3] at  
map at <console>:25
```

6) 将rdd3中元组按照key进行累加，统计每个单词出现的次数，生成rdd4。

```
val rdd4 = rdd3.reduceByKey(_+_)
```

回车运行，结果显示为：

```
rdd4: org.apache.spark.rdd.RDD[(String, Int)] = ShuffledRDD[4] at reduc  
eByKey at <console>:25
```



2.2.1 Spark-shell下的实例

在shell中，运行第一个Spark实例，步骤为：

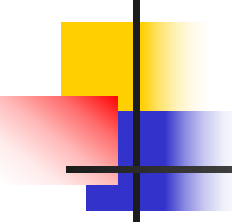
7) 输出rdd4中结果。输入：

```
rdd4.collect
```

回车运行，结果显示为：

```
res0: Array[(String, Int)] = Array((scala,1), (R,1), (python,1), (hello,2), (j  
ava,1), (spark,3), (hadoop,1))
```

注意：暂时不退出spark-shell，下一小节使用本实例介绍sparkWEB的使用。



2.2.2 SparkWEB的使用

每个Driver的SparkContext都会启动一个WEB界面，默认端口为4040，在浏览器中输入

localhost:4040

查看程序的诸多有用信息，包括：调度器Stage、Task列表；RDD大小和内存使用情况统计；环境配置信息；正在运行的执行器信息。

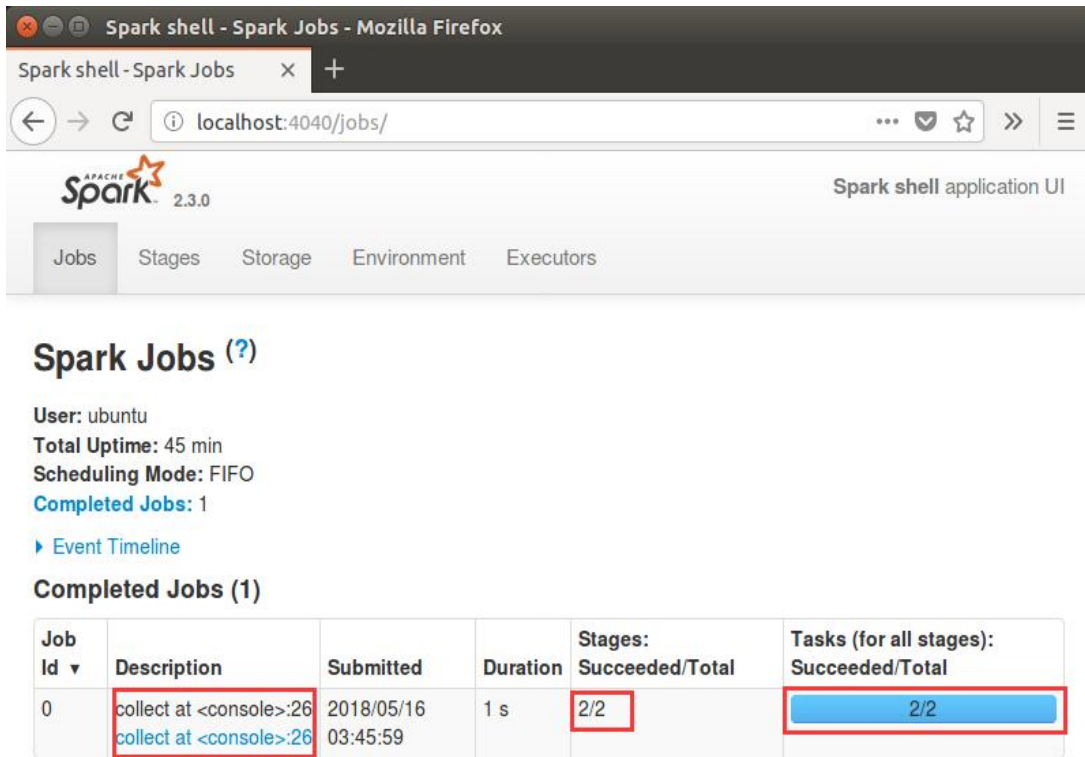
2.2.2 SparkWEB的使用

图示信息，前一小节的Job已完成，提交任务时间信息在

【Submitted】栏中显示，运行时间在【Duration】栏中显示。

【Stages】栏和【Tasks】栏中内容可知，任务被拆成了

2个Stage和2个Task。点击菜单栏中的【Stages】，进入任务Stages详情页面查看2个Stages的划分以及各自执行时间等信息。



The screenshot shows the Spark shell application UI in a Mozilla Firefox browser window. The address bar shows the URL `localhost:4040/jobs/`. The UI includes a navigation bar with tabs for Jobs, Stages, Storage, Environment, and Executors. The main content area displays the 'Spark Jobs' section, showing user information, total uptime, scheduling mode, and a list of completed jobs. A table titled 'Completed Jobs (1)' lists the job details, including Job ID, Description, Submitted time, Duration, Stages, and Tasks. The 'Submitted' column shows the time '2018/05/16 03:45:59'. The 'Duration' column shows '1 s'. The 'Stages' column shows '2/2'. The 'Tasks (for all stages): Succeeded/Total' column shows '2/2'.

Spark shell - Spark Jobs - Mozilla Firefox

Spark shell - Spark Jobs x +

localhost:4040/jobs/

APACHE Spark 2.3.0 Spark shell application UI

Jobs Stages Storage Environment Executors

Spark Jobs (?)

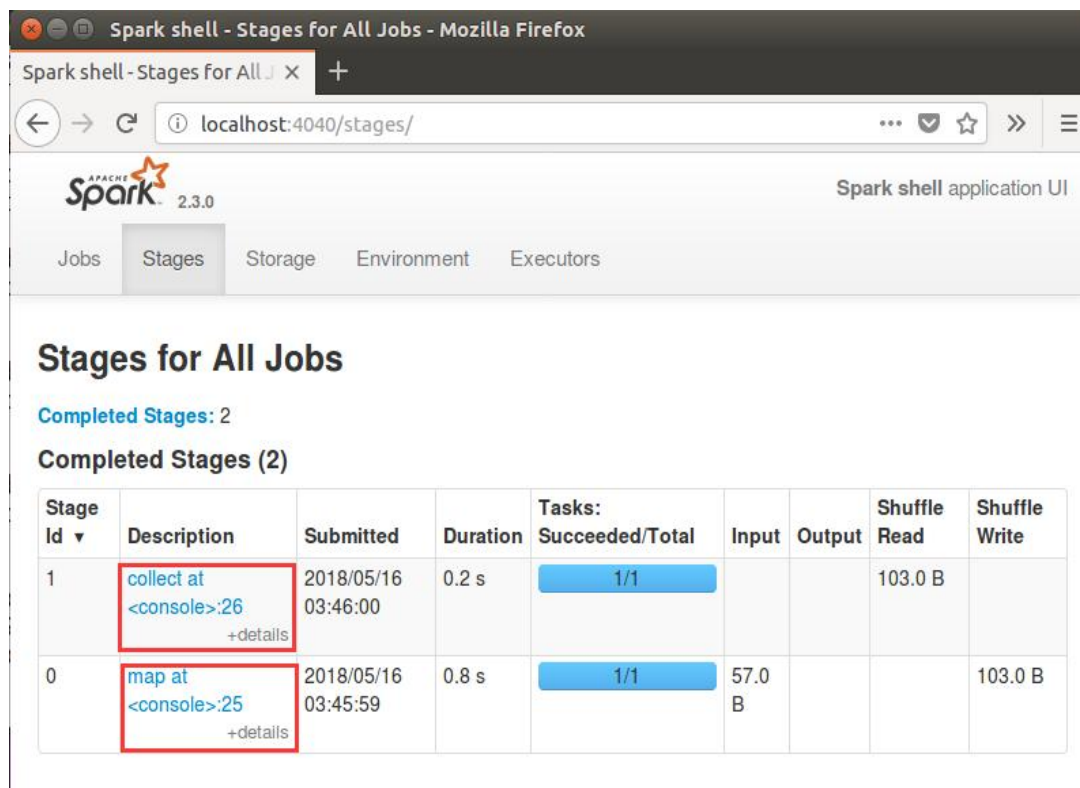
User: ubuntu
Total Uptime: 45 min
Scheduling Mode: FIFO
Completed Jobs: 1
[Event Timeline](#)

Completed Jobs (1)

Job Id	Description	Submitted	Duration	Stages: Succeeded/Total	Tasks (for all stages): Succeeded/Total
0	collect at <console>:26 collect at <console>:26	2018/05/16 03:45:59	1 s	2/2	2/2

2.2.2 SparkWEB的使用

任务Stages详情页面查看2个Stages的划分以及各自执行时间等信息。



Spark shell - Stages for All Jobs - Mozilla Firefox

Spark shell - Stages for All Jobs +

localhost:4040/stages/

APACHE Spark 2.3.0 Spark shell application UI

Jobs Stages Storage Environment Executors

Stages for All Jobs

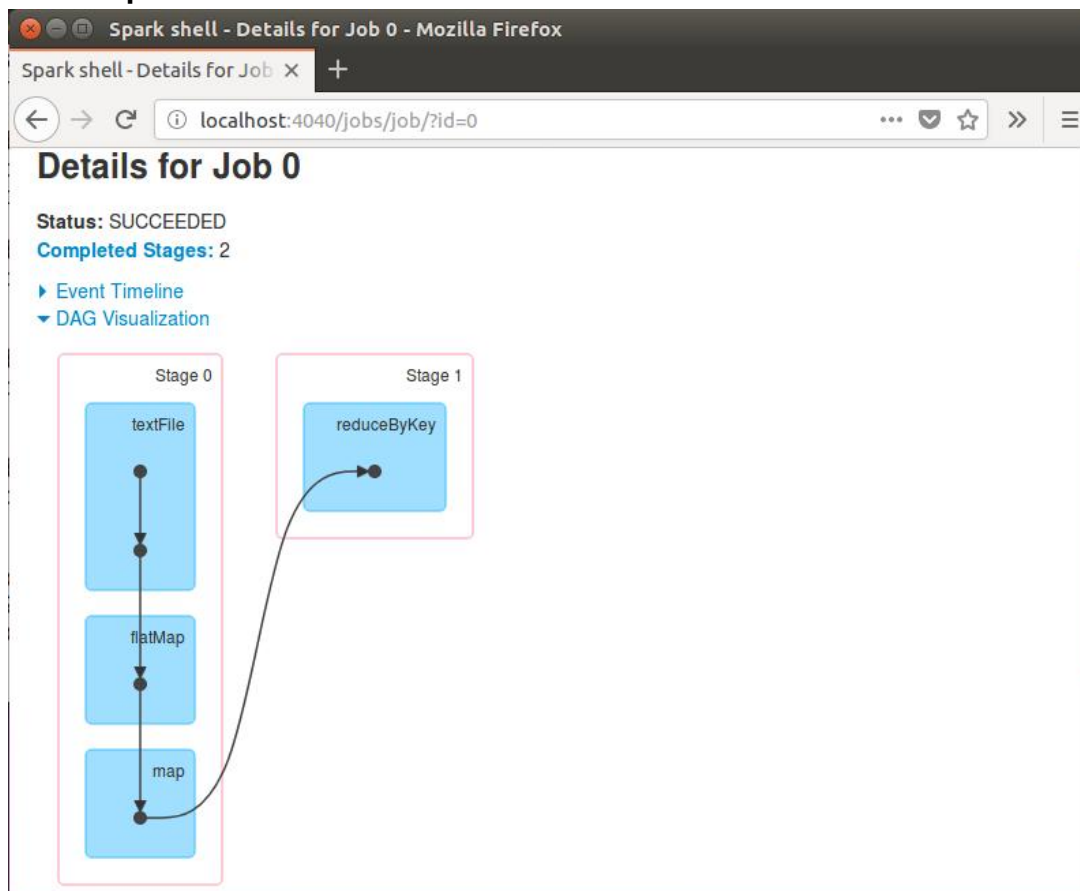
Completed Stages: 2

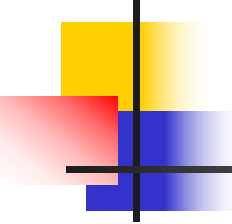
Completed Stages (2)

Stage Id ▾	Description	Submitted	Duration	Tasks: Succeeded/Total	Input	Output	Shuffle Read	Shuffle Write
1	collect at <console>:26 +details	2018/05/16 03:46:00	0.2 s	1/1			103.0 B	
0	map at <console>:25 +details	2018/05/16 03:45:59	0.8 s	1/1	57.0 B			103.0 B

2.2.2 SparkWEB的使用

点击图中【Description】栏中的链接，查看Job执行的细节

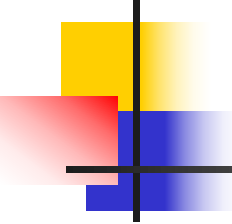




2.3 IDEA

为了开发Spark应用程序，需要安装有效的集成开发环境。IDEA是一个通用的集成开发环境，Spark应用通常采用Scala语言进行开发，而IDEA提供了最佳的Scala语言集成开发环境。本节将详细介绍Scala与IDEA的配置安装，以及使用IDEA创建一个Spark应用程序。最后，将介绍使用IDEA打包Spark应用程序，并通过spark-submit提交Spark环境运行。

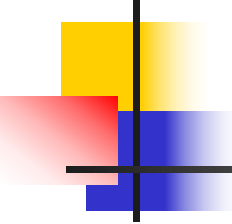
安装IDEA仅需要下载并解压IDEA的压缩包。安装完成IDEA后需要在IDEA中配置JDK、Scala以及安装Scala插件。



2.3.1 安装IDEA

前期准备：下载Scala和IDEA的安装包。具体版本号和官方下载地址如表所示。

	版本号	下载地址
Scala	Scala-2.11.8（ <u>与spark-shell显示的scala版本号一致</u> ）	https://downloads.lightbend.com/scala/2.11.8/scala-2.11.8.tgz
IDEA	Community 2018.1.6 for Linux	https://download.jetbrains.com/idea/ideaIC-2018.1.6.tar.gz

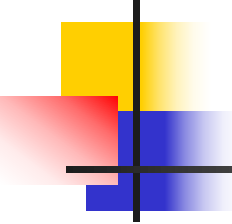


2.3.1 安装IDEA

1.Scala

Scala的配置过程分为三步：

- 1) 解压安装包；
- 2) 配置环境变量
- 3) 检查安装状态。

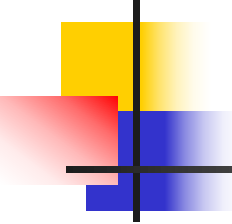


2.3.1 安装IDEA

1) 解压安装包，安装Scala。

在Scala安装包存放的目录下鼠标右键点击【Open in Terminal】，打开终端，将Scala压缩包解压到/usr/local目录下，命令为：

```
sudo tar zxvf scala-2.11.8.tgz -C /usr/local
```



2.3.1 安装IDEA

2) 配置环境变量。

终端中输入：**sudo gedit ~/.bashrc**。执行命令，在.bashrc文档最后加入字段：

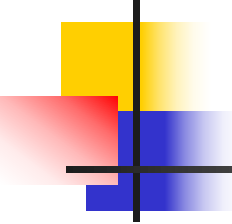
```
#SCALA
```

```
export SCALA_HOME=/usr/local/scala-2.11.8
```

```
export PATH=$PATH:$SCALA_HOME/bin
```

注意：SCALA_HOME的路径需与Scala解压后存放路径一致。

添加字段以后，点击【**Save**】保存修改。接下来在终端中输入命令使配置生效：**source ~/.bashrc**。

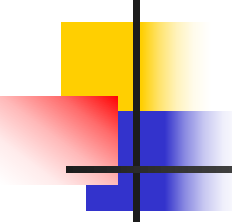


2.3.1 安装IDEA

3) 检查Scala安装状态。

终端中输入：`scala -version`。终端中出现如图所示的Scala的版本信息，则说明配置成功。

```
ubuntu@ubuntu:~$ scala -version
Scala code runner version 2.11.8 -- Copyright 2002-2016, LAMP/EPFL
ubuntu@ubuntu:~$
```

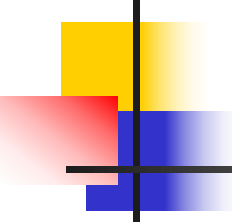


2.3.1 安装IDEA

2.IDEA

安装IDEA主要分为3步：

- 1) 解压安装包;
- 2) 启动IDEA进行初始化配置;
- 3) 安装Scala插件以及设置JDK和Scala依赖。



2.3.1 安装IDEA

1) 解压IDEA安装包

在IDEA安装包的存放目录下鼠标右键点击【Open in Terminal】，在终端中输入命令，将IDEA安装包解压到/usr/local目录下：

```
sudo tar zxvf ideaIC-2018.1.6.tar.gz -C /usr/local
```

2) 启动IDEA并进行初始化配置

在IDEA安装目录的bin目录下打开新的终端并输入： `./idea.sh`

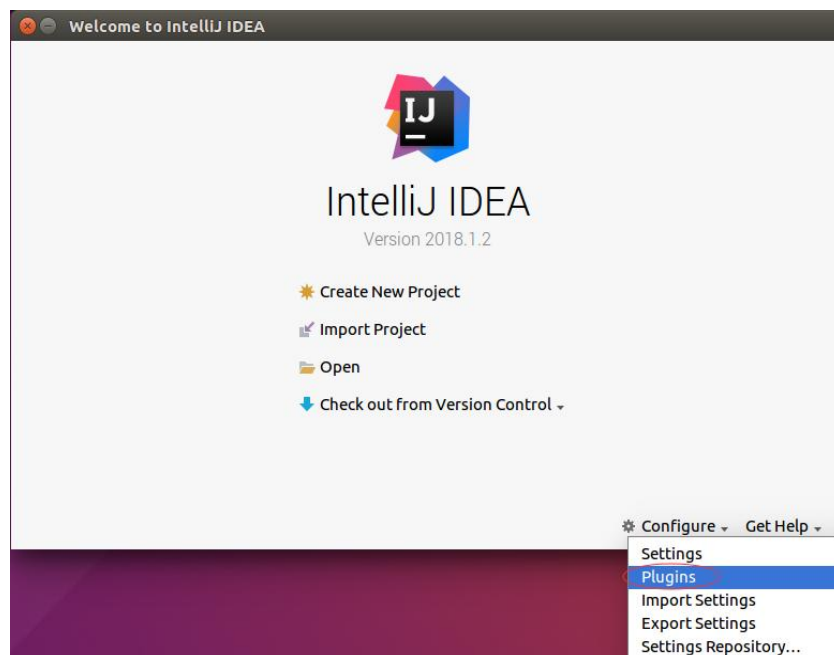
第一次启动IDEA时，弹窗中选择是否导入配置文件，选择默认的【Do not import settings】。接下来，在用户协议弹窗中，点击【Accept】，进入UI主题选择界面。点击【Skip Remaining and Set Defaults】，使用默认设置。

2.3.1 安装IDEA

3) 安装Scala插件并设置JDK与Scala SDK的位置

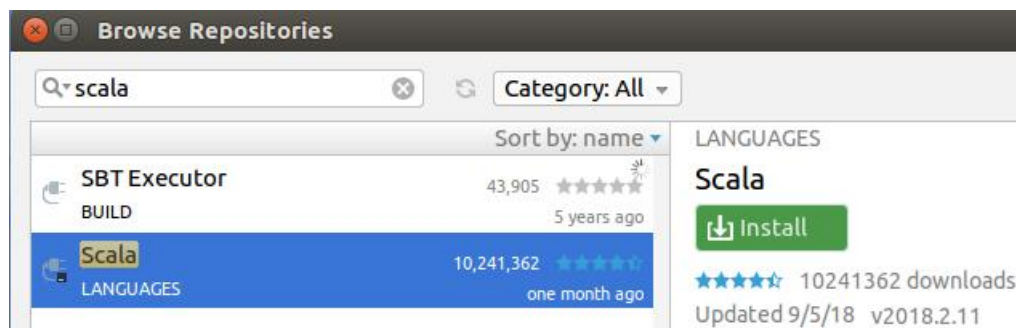
①安装Scala插件。具体步骤为：

第一步，点击初始界面中的【Configure】，然后点击【Plugins】



2.3.1 安装IDEA

第二步，在新窗口中搜索Scala插件，点击【Install】进行安装，如图所示。自动匹配下载安装当前IDEA版本对应的Scala插件。安装完成后重启IDEA即可使用Scala插件。



注意：Scala插件下载长时间无响应或者搜索失败（弹窗显示下载失败）的解决方案：在官网中下载Scala插件（<http://plugins.jetbrains.com/plugin/1347-Scala>）进行本地安装。参考下一页操作。

2.3.1 安装IDEA

安装scala插件失败：手动下载Scala插件进行安装。根据IDEA安装包名称中包含版本号信息下载Scala插件，例如本书中使用的IDEA安装名为ideaIC-2018.1.2.tar.gz，版本号为2018.1.2。在官

网搜索Scala插件版本如右图所示，根据【Compatibility with IntelliJ IDEA Community】显示的版本支持信息，选择包含安装IDEA版本号的Scala插件，点击【Download】下载。

Version History

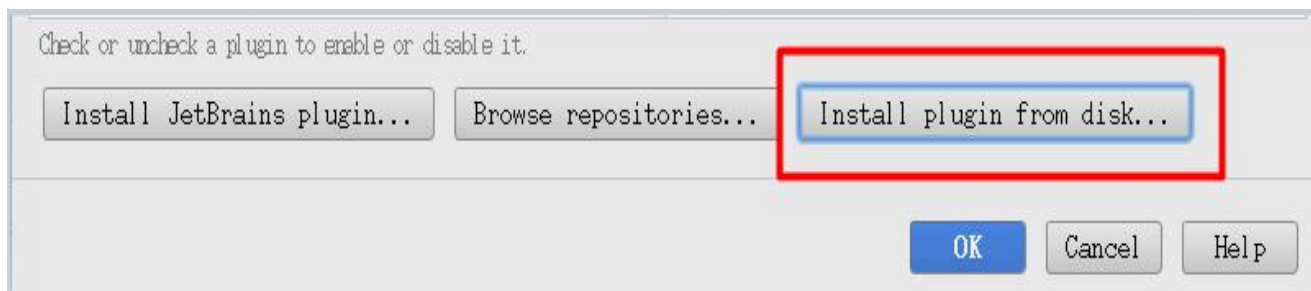
[Stable](#)[EAP](#)[Nightly](#)

Version	Compatibility with IntelliJ IDEA Community	Update date	
2019			
2019.1.6	2019.1 (rc)	Mar 25, 2019	Download
2019.1.5	2019.1 (rc)	Mar 21, 2019	Download
2019.1.3	2019.1 (rc)	Mar 14, 2019	Download
2018.3.7	2018.3 — 2018.3.6	Mar 12, 2019	Download
2019.1.2	2019.1 (rc)	Mar 06, 2019	Download
2018.3.6	2018.3 — 2018.3.6	Feb 01, 2019	Download
2019.1.1	2019.1 (rc)	Jan 11, 2019	Download
2018			
2018.3.5	2018.3 — 2018.3.6	Dec 19, 2018	Download

2.3.1 安装IDEA

手动下载Scala插件并进行安装的步骤为：

点击IDEA初始页面中【Configure】，然后点击【Plugins】重新打开插件搜索窗口。如下图所示，点击【Install plugin from disk】按钮，找到Scala插件存放路径，点击【OK】，安装完成后根据提示，重启IDEA。

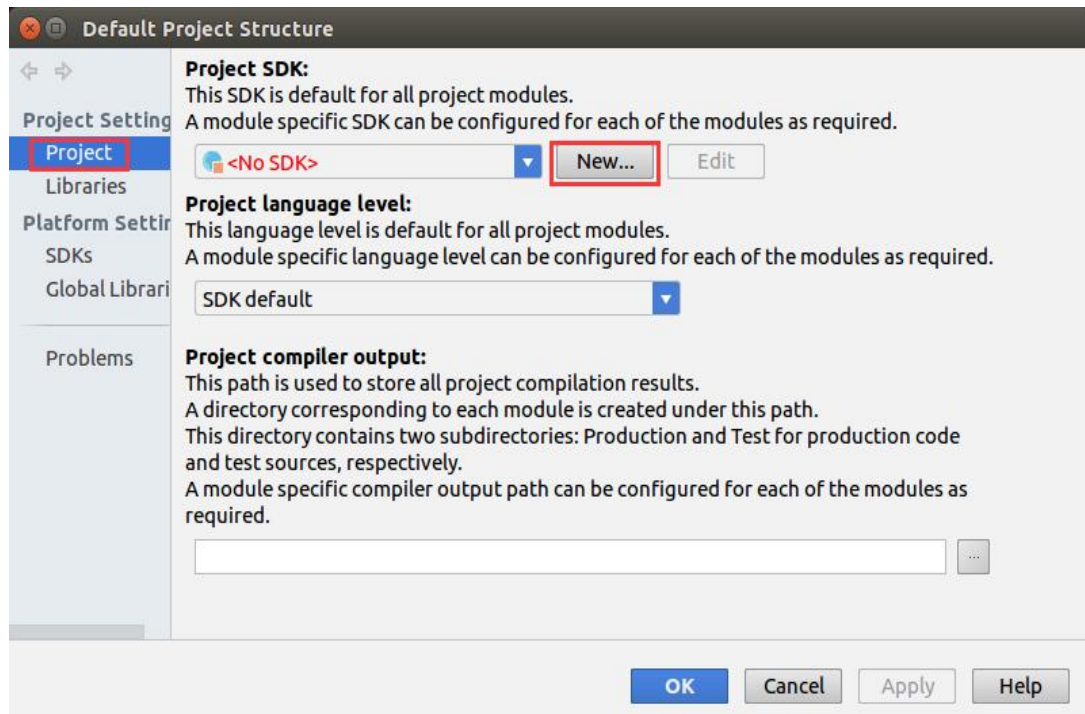


2.3.1 安装IDEA

②设置全局JDK

第一步，在IDEA初始界面点击右下角的【Configure】，然后点击【Project Defaults】扩展菜单中的【Project Structure】；

第二步，点击弹出窗口左侧的【Project】，然后点击【New...】，找到并选择JDK的安装路径（例如“/usr/local/jdk1.8.0_171”），点击【OK】完成全局JDK的设置，如图所示。



2.3.1 安装IDEA

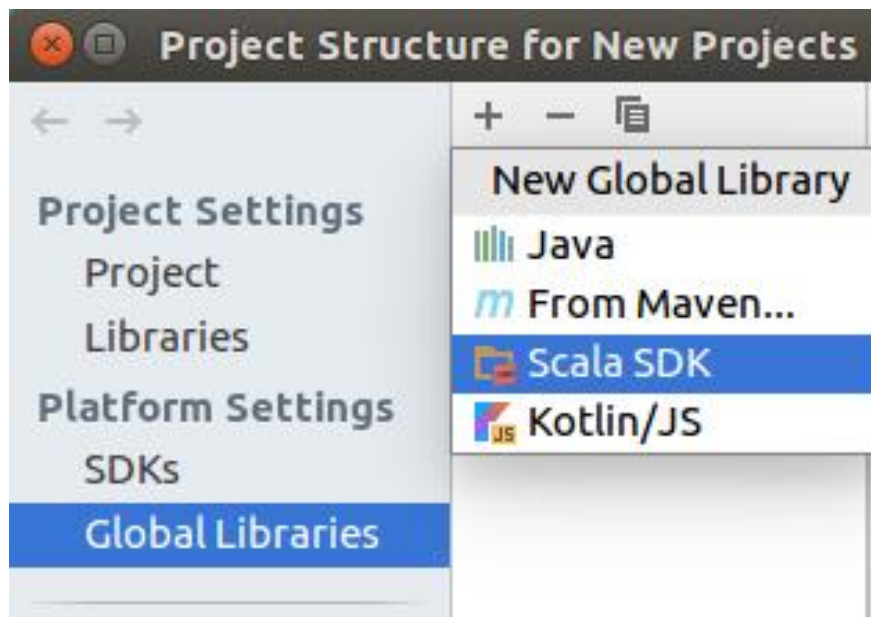
③设置全局Scala SDK

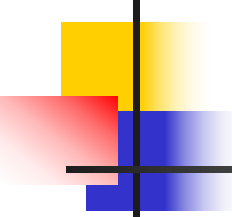
第一步，选择左侧的

【Global Libraries】，点击【+】，
选择下拉菜单中的【Scala SDK】；

第二步，在弹出窗口中，点击
弹窗窗口中的【Browse...】，定位

到Scala的安装位置（本书为“/usr/local/scala-2.11.8”），点击【OK】完成
全局Scala SDK的设置。如果点击【Scala SDK】无响应，建议先新建项目，
然后设置Scala SDK。





2.3.2 IDEA的实例（Scala）

本小节将介绍如何使用**IDEA**创建一个**Scala**项目，并运行**WordCount**程序。

- 1.新建项目
- 2.设置项目名、JDK和Scala SDK
- 3.导入Spark的jars文件夹
- 4.编写代码
- 5.运行程序

2.3.2 IDEA的实例（Scala）

1.新建项目

在初始页面上点击

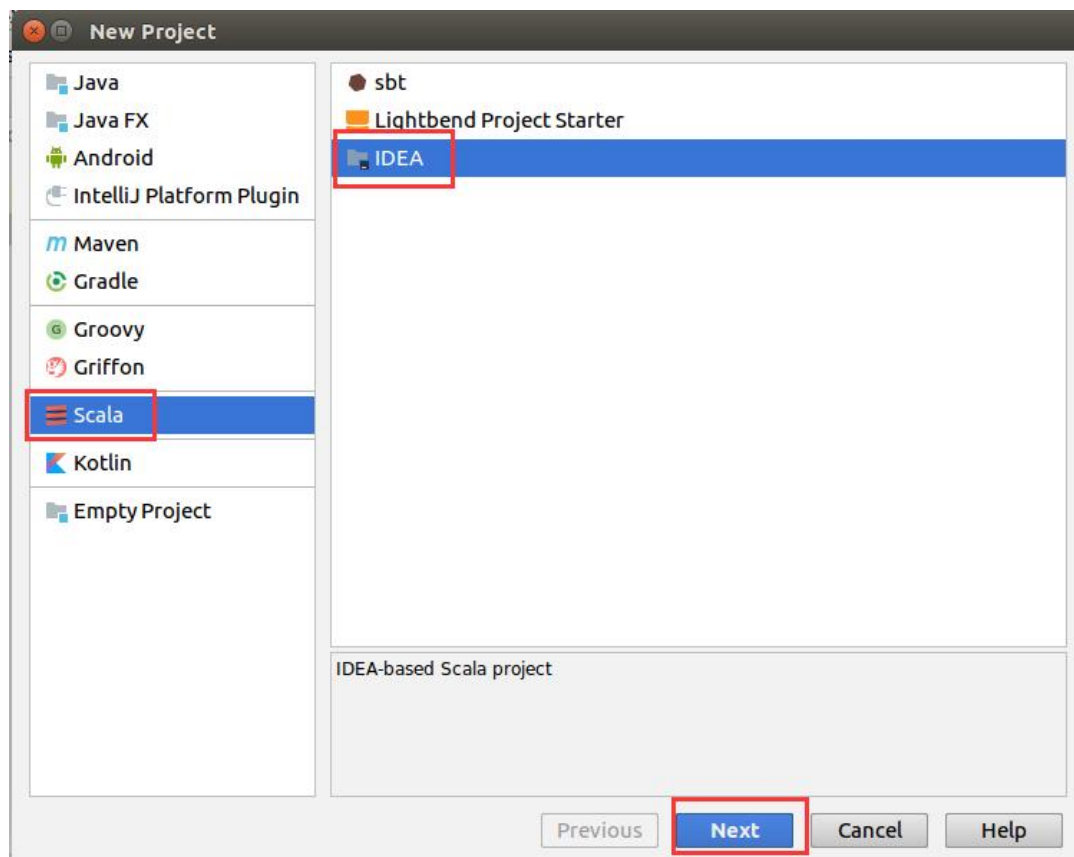
【Create New Project】，

选择弹出窗口左侧框中的

【Scala】，接下来选中

右侧框中的【IDEA】，

并点击【Next】。

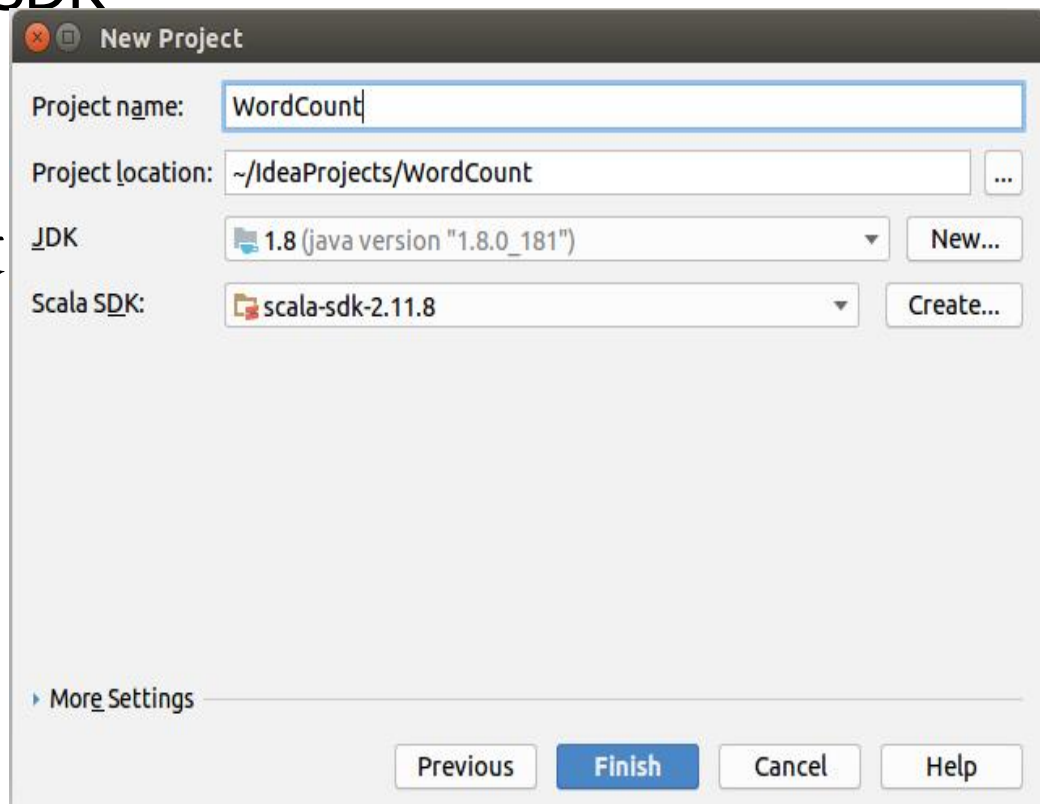


2.3.2 IDEA的实例（Scala）

2.设置项目名、JDK和Scala SDK

IDEA自动导入全局JDK

和Scala SDK依赖，只需要设置项目名称，如图所示。



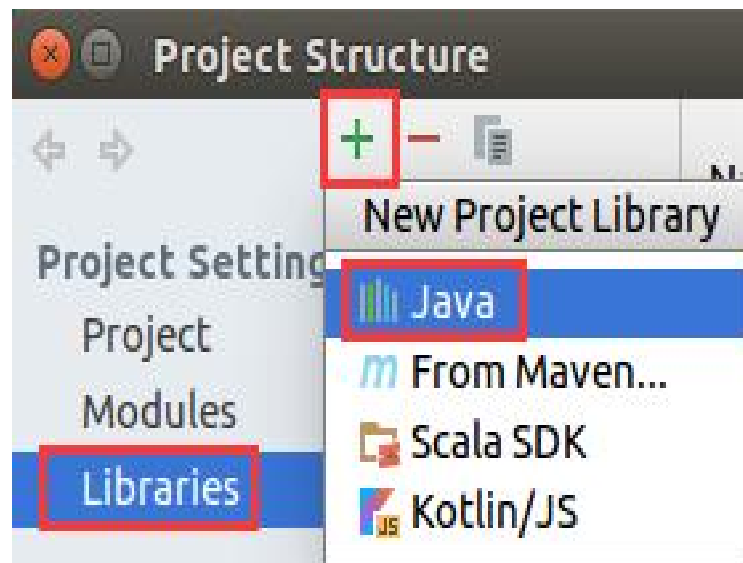
2.3.2 IDEA的实例（Scala）

3. 导入Spark的jars文件夹

具体步骤为：

（1）点击菜单栏中的【File】，选择下拉菜单中的【Project Structure】，进入环境配置界面。

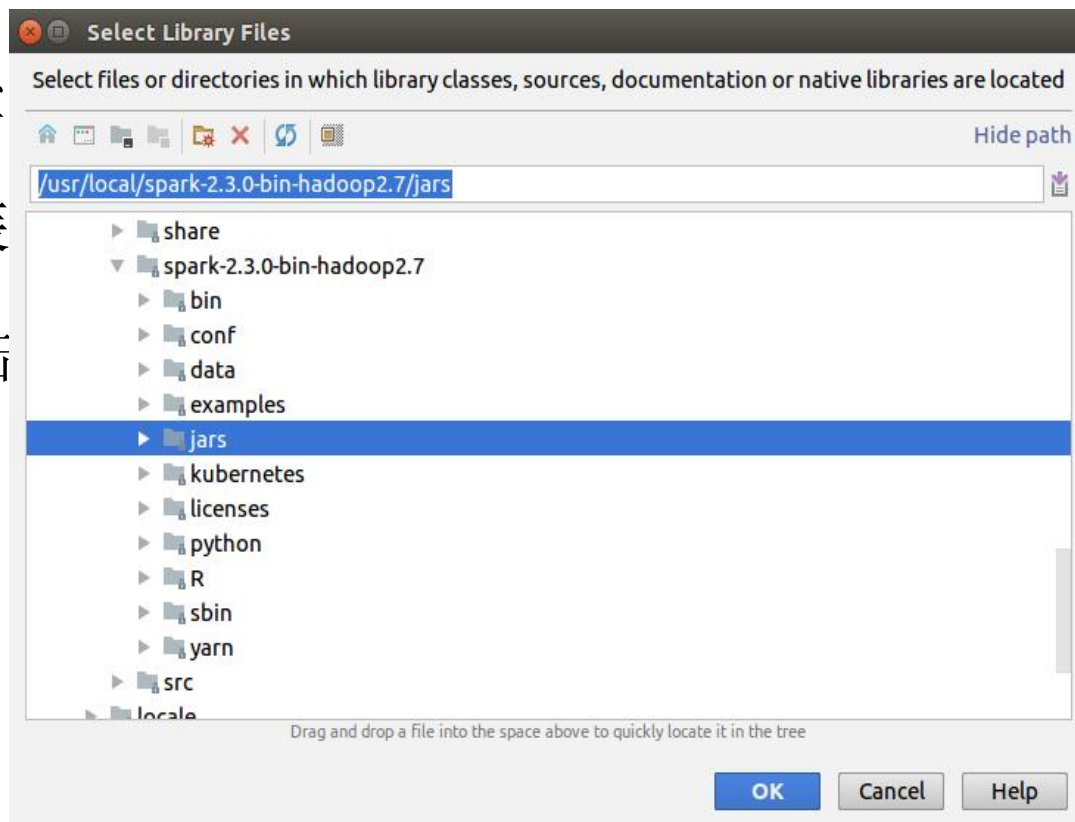
（2）选择左侧框中的【Libraries】，再点击【+】，选择下拉菜单中的【Java】，如图所示。



2.3.2 IDEA的实例（Scala）

3. 导入Spark的jars文件夹

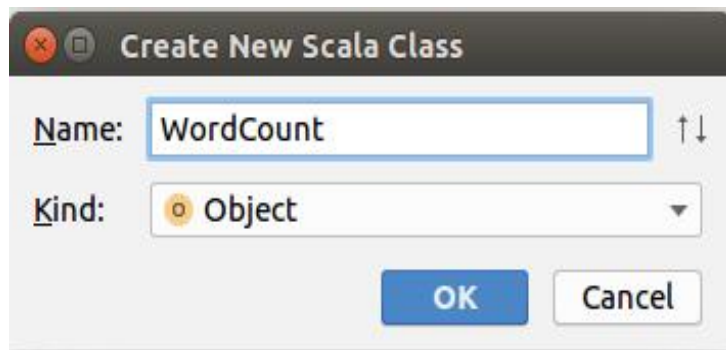
（3）在如图所示的路径选择窗口中，选中Spark安装目录下的jars文件夹，并点击【OK】完成Spark项目的jar依赖包导入。

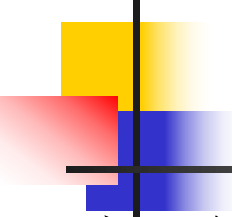


2.3.2 IDEA的实例（Scala）

4.编写代码

在WordCount/src文件夹下，新建一个Scala项目文件。右键点击src文件夹，在窗口中，选择【New】，然后选择【Scala Class】，出现图示窗口。将【Kind】设置为【Object】，并在【Name】文本框中，输入“WordCount”作为文件名。



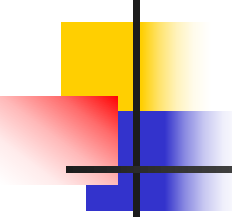


2.3.2 IDEA的实例（Scala）

4.编写代码。在新建的WordCount.scala中输入代码：

```
import org.apache.spark.{SparkConf,SparkContext}
object WordCount {
    def main(args: Array[String]) {
        val conf = new SparkConf().setAppName("mySpark").setMaster("local")
        val sc = new SparkContext(conf)
        val rdd = sc.textFile(args(0))
        val wordcount = rdd.flatMap(_.split("\t")).map((_,1)).reduceByKey(_ + _)
        for(arg <- wordcount.collect())
            print(arg + " ")
            println()
        sc.stop() }}
```

注意：代码中，`args(0)`表示输入文件路径。需要在运行项目时，设置参数值。



2.3.2 IDEA的实例（Scala）

5.运行程序

运行程序的完整步骤包括：编译代码、调试配置、运行程序、查看结果

（1）编译代码

点击菜单栏中的【Build】，然后点击【Build Project】，或者使用“**Ctrl+F9**”组合键对代码进行编译，编译结果会在Event Log中显示

（2）调试配置

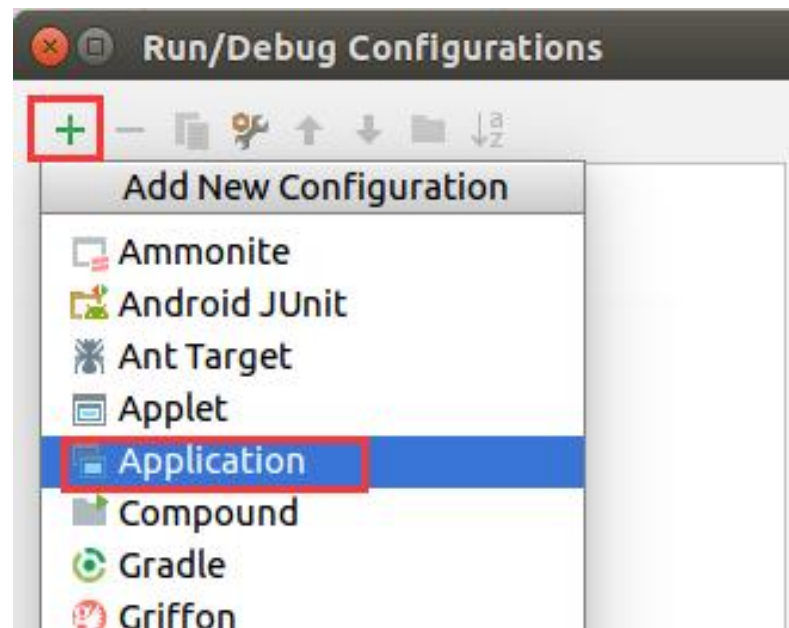
编译通过以后，单击菜单【Run】，然后点击【Edit Configuration】打开配置窗口。在新弹出的【Run/Debug Configuration】窗口中，添加一个【Application】。

2.3.2 IDEA的实例（Scala）

5.运行程序

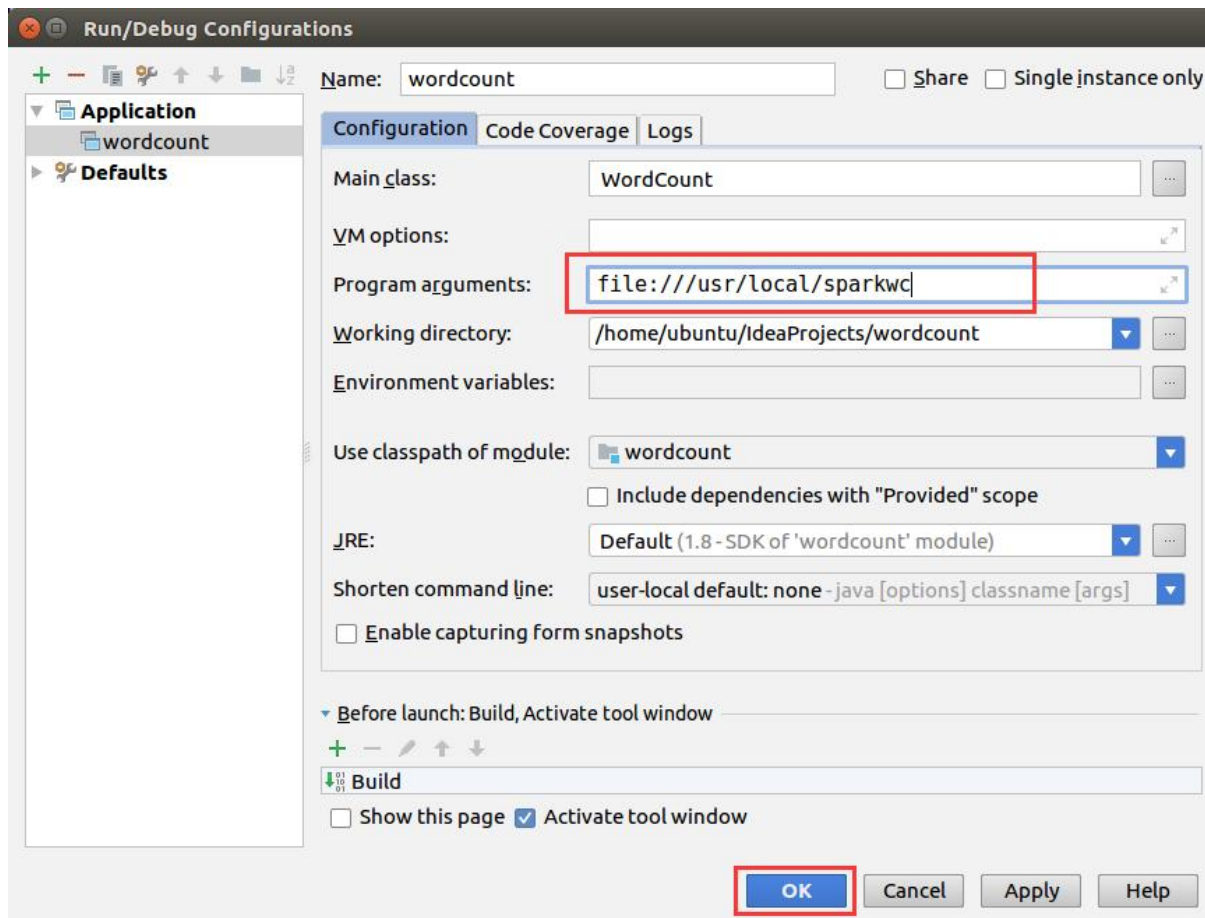
（2）调试配置。具体步骤为：

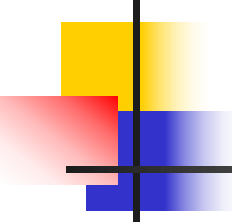
- 1) 点击【+】创建一个【Application】；
- 2) 在【Name】输入框中输入“wordcount”作为名称；
- 3) 设置【Main Class】的路径，点击右侧的【...】，选择项目中的“WordCount.scala”路径；
- 4) 在【Program arguments】栏中输入参数 <file:///usr/local/sparkwc>（wordcount程序要读取的文件路径），并点击【OK】。



2.3.2 IDEA的实例（Scala）

5.运行程序





2.3.2 IDEA的实例（Scala）

5.运行程序

（3）运行程序

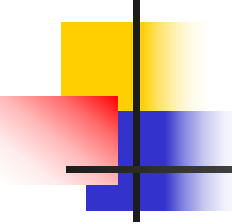
在代码空白处，点击鼠标右键，选择弹出菜单中的

【Run 'wordcount'】。

（4）查看结果

在**IDEA**界面下方的控制台中，出现运行结果：

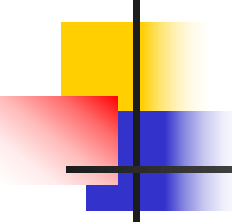
(scala,1) (spark,3) (hadoop,1) (R,1) (python,1) (hello,2) (java,1)



2.3.3 IDEA打包运行（Scala）

在上一小节创建的WordCount项目基础上，本小节将介绍使用IDEA打包项目和使用spark-submit提交jar包到Spark运行的流程：

1. 生成JAR包。
2. 将jar包复制到Spark的根目录下
3. 使用spark-submit命令运行jar包



2.3.3 IDEA打包运行（Scala）

1. 生成JAR包。

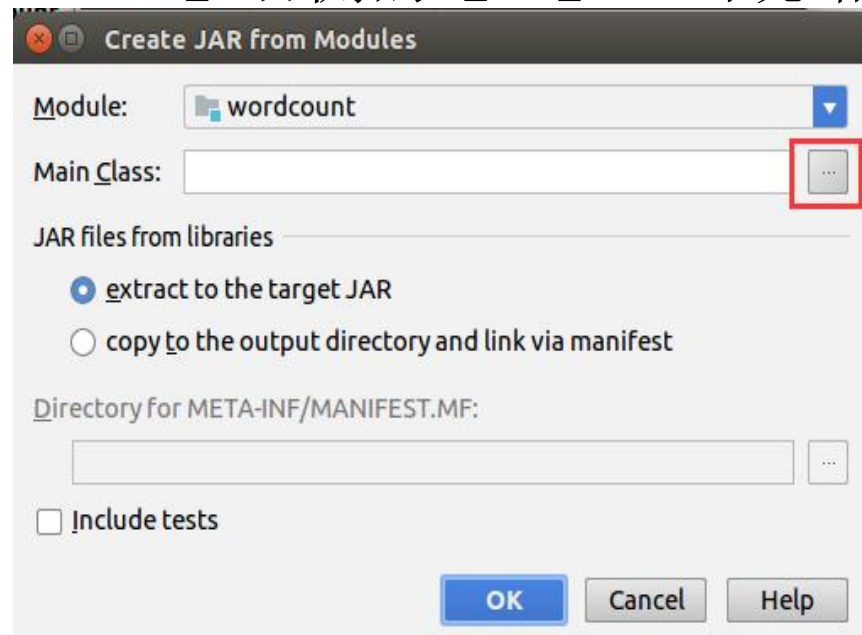
（1）点击菜单栏【File】，然后点击【Project Structure】，进入项目配置界面。

（2）选择【Artifacts】，然后在界面的中间操作界面中点击【+】，选择【JAR】菜单中的【From modules with dependencies】。

2.3.3 IDEA打包运行（Scala）

1. 生成JAR包。

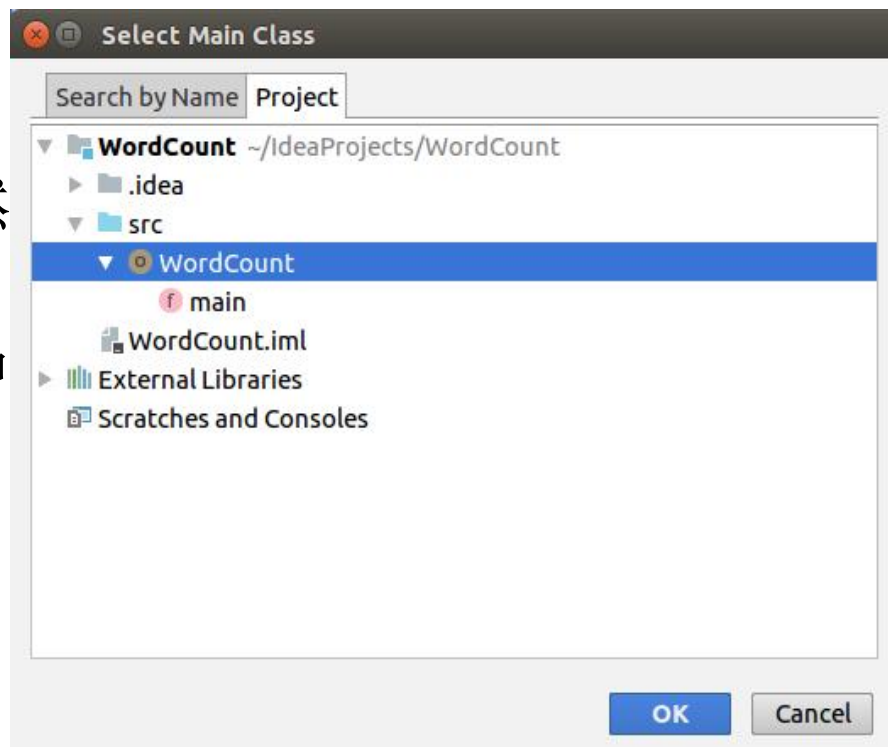
（3）点击【Main Class】右侧的【...】，出现路径选择窗口。



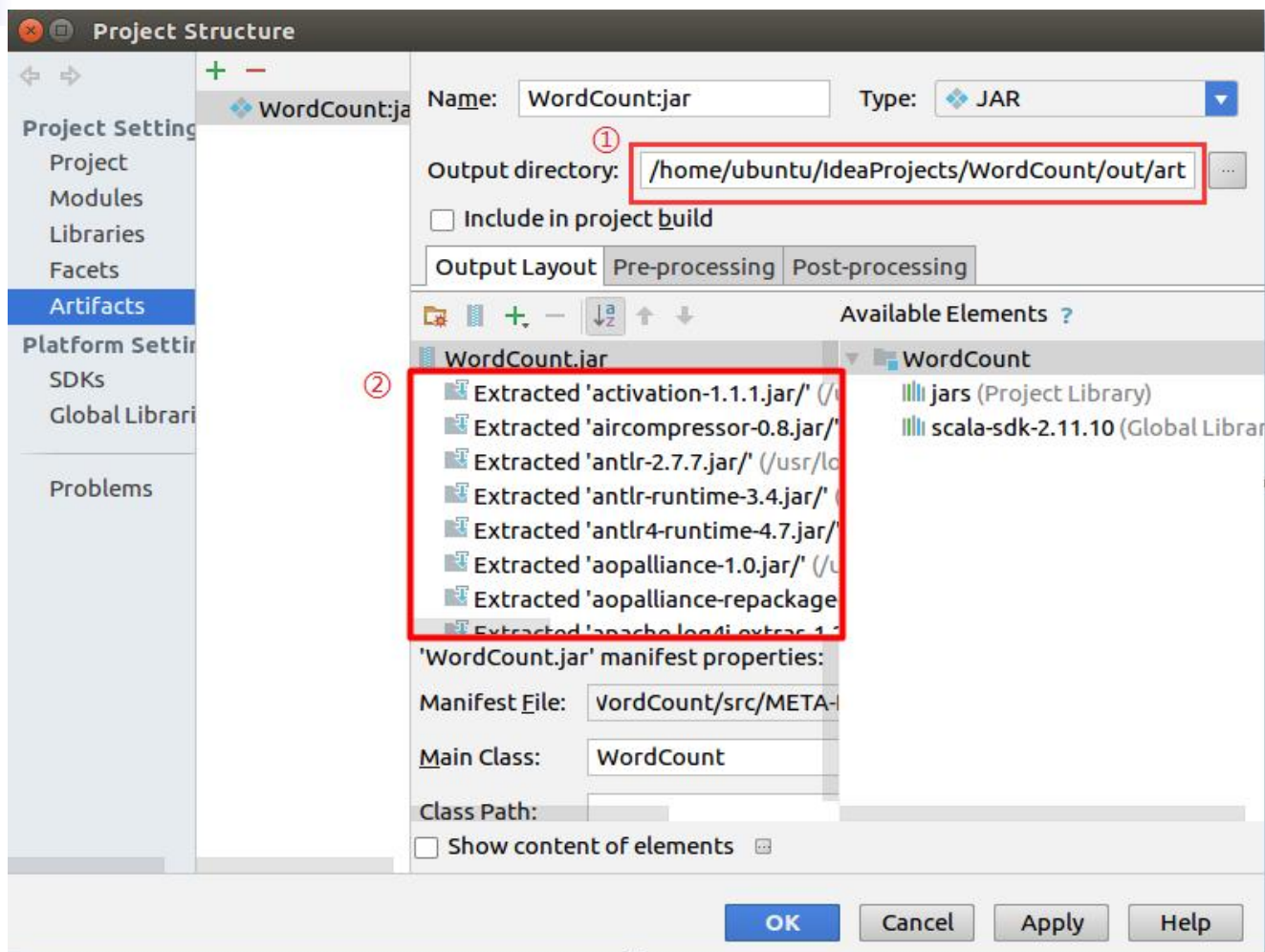
2.3.3 IDEA打包运行（Scala）

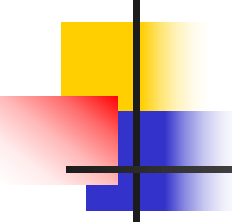
1. 生成JAR包。

（4）选择图中的【Project】，然后在项目WordCount目录下，选中src文件夹下的WordCount.scala并点击【OK】。



2.3.3 IDEA打包运行（Scala）





2.3.3 IDEA打包运行（Scala）

1. 生成JAR包。

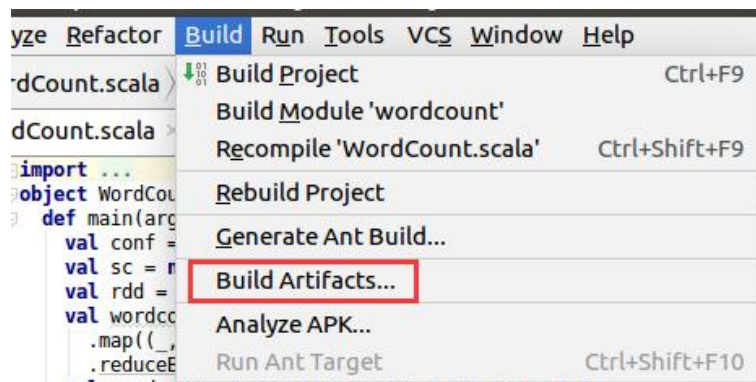
项目配置完成，在图中①号框所示【Output directory】处，修改jar包的保存位置。删除图中②号框内的多余jar包，仅保留【WordCount.jar】以及【'WordCount' Compile output】。使用全选快捷键“Ctrl+A”选中②号框内全部选项，然后使用“Ctrl+鼠标左键”进行反选。再点击②号框上方的【-】删除其余被选中的jar包。

注意：生成jar包之前，删除代码中setMaster("local")。

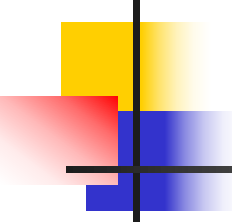
2.3.3 IDEA打包运行（Scala）

1. 生成JAR包。

（5）点击菜单栏的【Build】，然后点击【Build Artifacts】生成项目的打包文件，如图所示。在弹出界面中，进一步选择【Build】。



生成的jar包默认位置在项目文件夹out目录下的artifacts文件夹中。本书中，路径为“/home/ubuntu/IdeaProjects/wordcount/out/artifacts/WordCount.jar”。

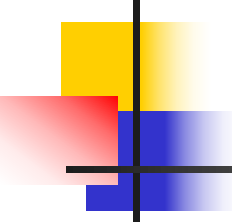


2.3.3 IDEA打包运行（Scala）

2. 将jar包复制到Spark的根目录下

在jar包所在目录下，打开终端并输入命令：

```
sudo cp WordCount.jar /usr/local/spark-2.3.0-bin-hadoop2.7
```

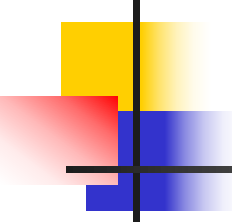


2.3.3 IDEA打包运行（Scala）

3. 使用spark-submit命令运行jar包

通过spark-submit提交运行jar包的命令格式为：

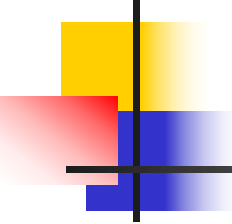
```
./bin/spark-submit  
  --class main-class           //需要运行的程序的主类，  
                                //应用程序的入口点  
  --master master-url          //Master URL，下面会有具体解释  
  --deploy-mode deploy-mode    //部署模式  
  ... # other options  
  application-jar              //应用程序JAR包  
  [application-arguments]     //传递给主类的主方法的参数
```



2.3.3 IDEA打包运行（Scala）

3. 使用spark-submit命令运行jar包

本小节的实例中，**main-class**为WordCount，**master-url**为local，**deploy-node**和other options缺省，**application-jar**为WordCount.jar，**[application-arguments]**为单词计数文件的存放路径：file:///usr/local/sparkwc。



2.3.3 IDEA打包运行（Scala）

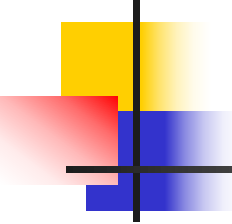
3. 使用spark-submit命令运行jar包

将本小节生成的jar包提交spark-submit运行的步骤为：

（1）在Spark的安装目录下（本书为“/usr/local/ spark-2.3.0-bin-hadoop2.7”），打开一个终端并输入：

```
./bin/spark-submit --class WordCount --master local WordCount.jar file:///
```

```
usr/local/sparkwc
```

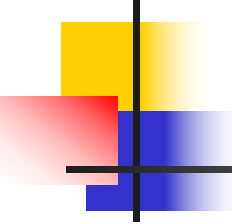


2.3.3 IDEA打包运行（Scala）

3. 使用spark-submit命令运行jar包

（2）终端中显示运行结果：

(scala,1) (spark,3) (hadoop,1) (R,1) (python,1) (hello,2) (java,1)

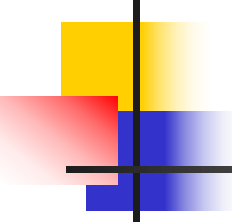


2.4 Eclipse

本节将主要介绍使用**Eclipse**编写**Spark**应用程序。**Eclipse**是一种常用的集成开发环境，也支持搭建**Scala**语言开发环境。本节将介绍**Eclipse**的配置安装，以及在**Eclipse**上创建**Spark**项目的方法。

特别说明：**Eclipse**目前不支持**Spark**项目打包。

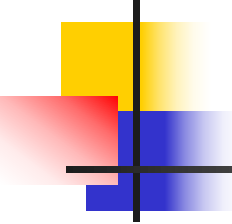
安装**Eclipse**的步骤和**IDEA**类似。第一步，下载并解压**Eclipse**安装包；第二步，下载并解压**Eclipse**的**Scala**插件安装包。



2.4.1 安装Eclipse

前期准备：下载 Eclipse和Scala IDE插件压缩包。具体版本号
和官方下载地址如表所示：

软件名	版本号	下载地址
Eclipse	Eclipse 4.7.3a	http://ftp.yz.yamagata-u.ac.jp/pub/eclipse/technology/epp/downloads/release/oxygen/3a/eclipse-jee-oxygen-3a-linux-gtk-x86_64.tar.gz
Scala IDE插件	4.7.0	http://download.scala-ide.org/sdk/lithium/e47/scala212/stable/update-site.zip



2.4.1 安装Eclipse

Eclipse安装步骤主要为：

1. 解压Eclipse安装包。

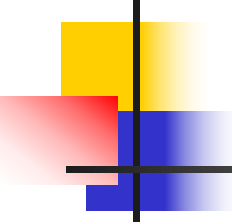
在Eclipse安装包的存放目录下，打开终端，输入命令：

```
sudo tar zxvf eclipse-jee-oxygen-3a-linux-gtk-x86_64.tar.gz -C /usr/local
```

2. 解压缩Scala插件压缩包update-site.zip到/usr/local目录下。解压命令为：

```
sudo unzip update-site.zip -d /usr/local
```

解压完成以后，在/usr/local目录下，出现base文件夹。base文件夹中有：features、plugins、artifacts.jar、content.jar、site.xml。



2.4.1 安装Eclipse

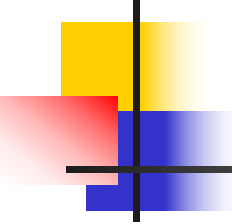
Eclipse安装步骤主要为：

3. 将features文件夹中的所有文件复制到Eclipse安装路径对应的features文件夹中。在/usr/local/base目录下，打开终端，输入命令：

```
sudo cp -R features/* /usr/local/eclipse/features
```

4. 将plugins文件夹中的所有文件复制到Eclipse安装路径的plugins文件夹中。输入命令：

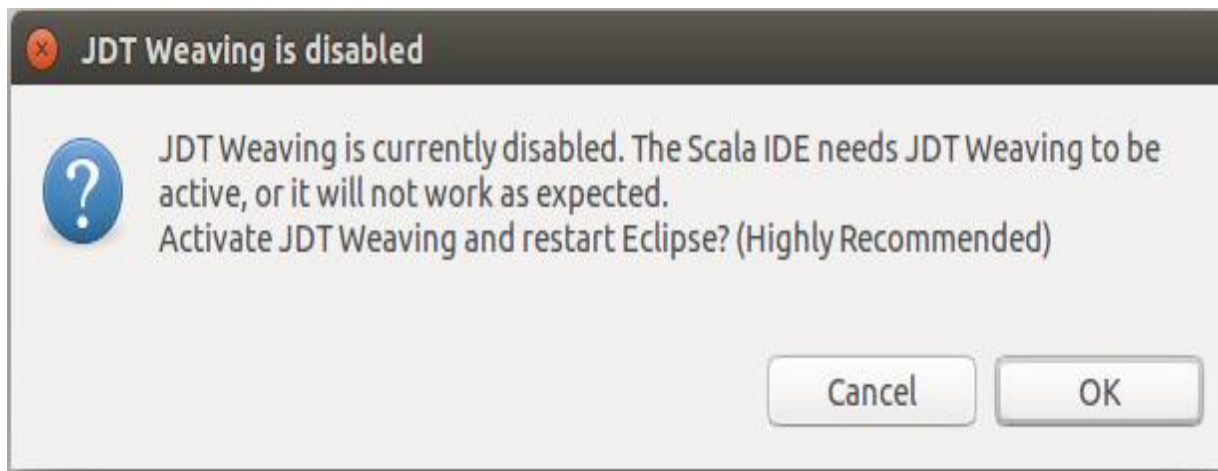
```
sudo cp -R plugins/* /usr/local/eclipse/plugins
```

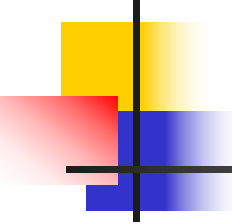


2.4.1 安装Eclipse

Eclipse安装步骤主要为：

5. 重新启动Eclipse。出现图示的弹窗，点击【OK】。完成安装。

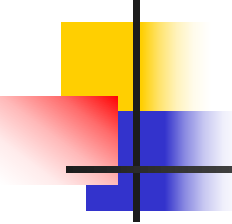




2.4.2 Eclipse的实例（Scala）

Eclipse进行Spark应用程序开发的步骤分为三步：

- 1.启动Eclipse;
- 2.建立项目，添加依赖;
- 3.编写程序;
- 4.编译并运行。



2.4.2 Eclipse的实例（Scala）

1.启动Eclipse

在Eclipse的安装目录下（本书为“/usr/local/eclipse”），打开终端，执行命令：**./eclipse**。出现设置工作空间（项目存放位置）的窗口。本书选择默认路径，也可以点击【Browse...】设置工作空间的路径。设置完成以后，点击【Launch】按钮进入Eclipse主界面。

2.4.2 Eclipse的实例（Scala）

2.新建项目，配置依赖

（1）依次点击菜单栏中的【File】>>【New】>>【Other】。或者

直接点击菜单栏下方的快捷按钮选项【New】。打开如图所示界面。

Select a wizard

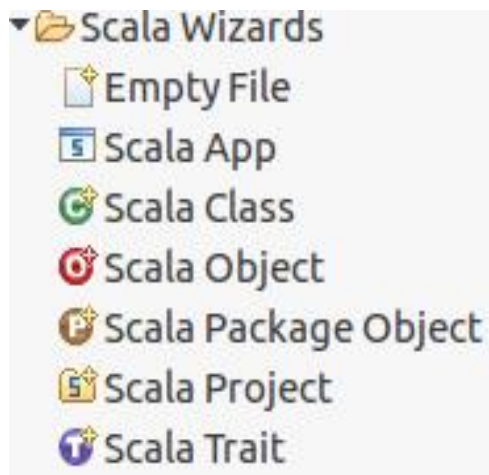
Wizards:

- ▶ Gradle
- ▶ Java
- ▶ Maven
- ▶ Oomph
- ▶ Plug-in Development
- ▶ Scala Wizards
- ▶ Tasks
- ▶ User Assistance
- ▶ XML
- ▶ Examples

2.4.2 Eclipse的实例（Scala）

2.新建项目，配置依赖

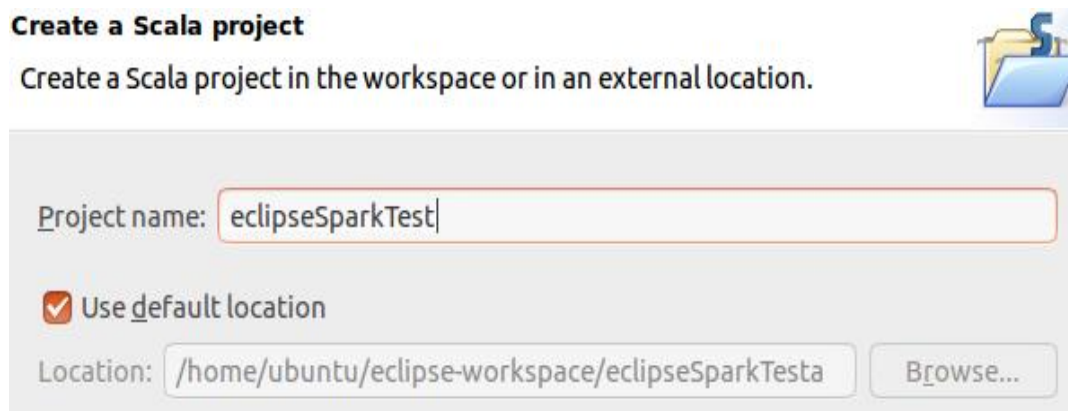
（2）点击【Scala Wizards】，出现如图所示选择页面。



2.4.2 Eclipse的实例（Scala）

2.新建项目，配置依赖

（3）选择【Scala Project】并点击【Next】进入如图所示的项目名称、项目存放位置等设置界面。



The screenshot shows the 'Create a Scala project' dialog box in the Eclipse IDE. The title bar reads 'Create a Scala project'. Below the title, it says 'Create a Scala project in the workspace or in an external location.' To the right of this text is a small icon of a folder with a blue 'S' on it. The main area of the dialog has a light gray background. It contains a text input field for 'Project name:' with the text 'eclipseSparkTest' entered. Below this is a checkbox labeled 'Use default location' which is checked. At the bottom, there is a 'Location:' label followed by a text input field containing the path '/home/ubuntu/eclipse-workspace/eclipseSparkTest'. To the right of this field is a 'Browse...' button.

Create a Scala project

Create a Scala project in the workspace or in an external location.

Project name: eclipseSparkTest

☒ Use default location

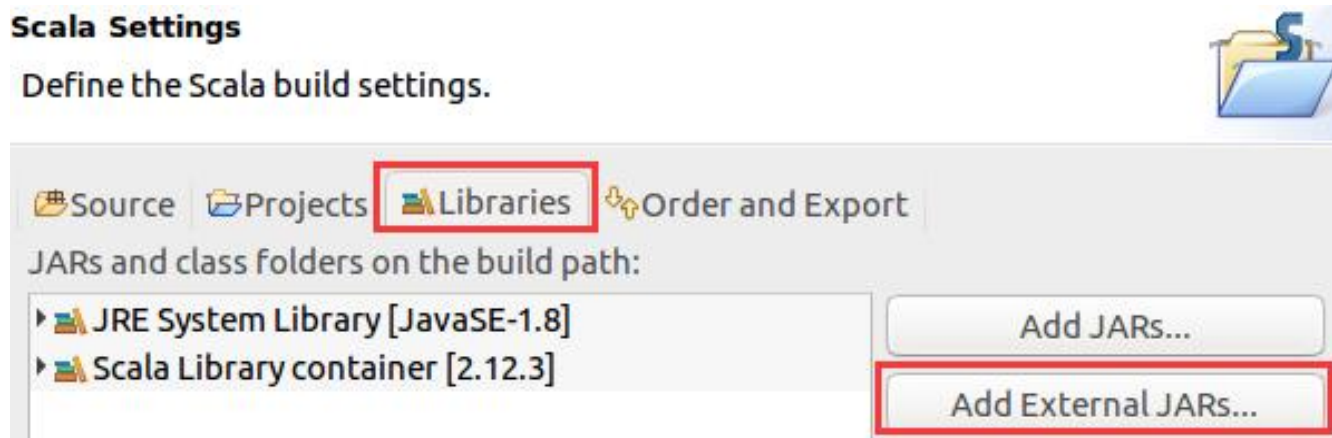
Location: /home/ubuntu/eclipse-workspace/eclipseSparkTest Browse...

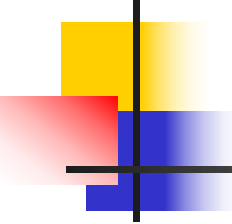
2.4.2 Eclipse的实例（Scala）

2.新建项目，配置依赖

（4）设置信息以后，点击【Next】，进入项目依赖jar包设置界面。选择【Libraries】，然后点击右侧的【Add External JARs...】，如图

所示：





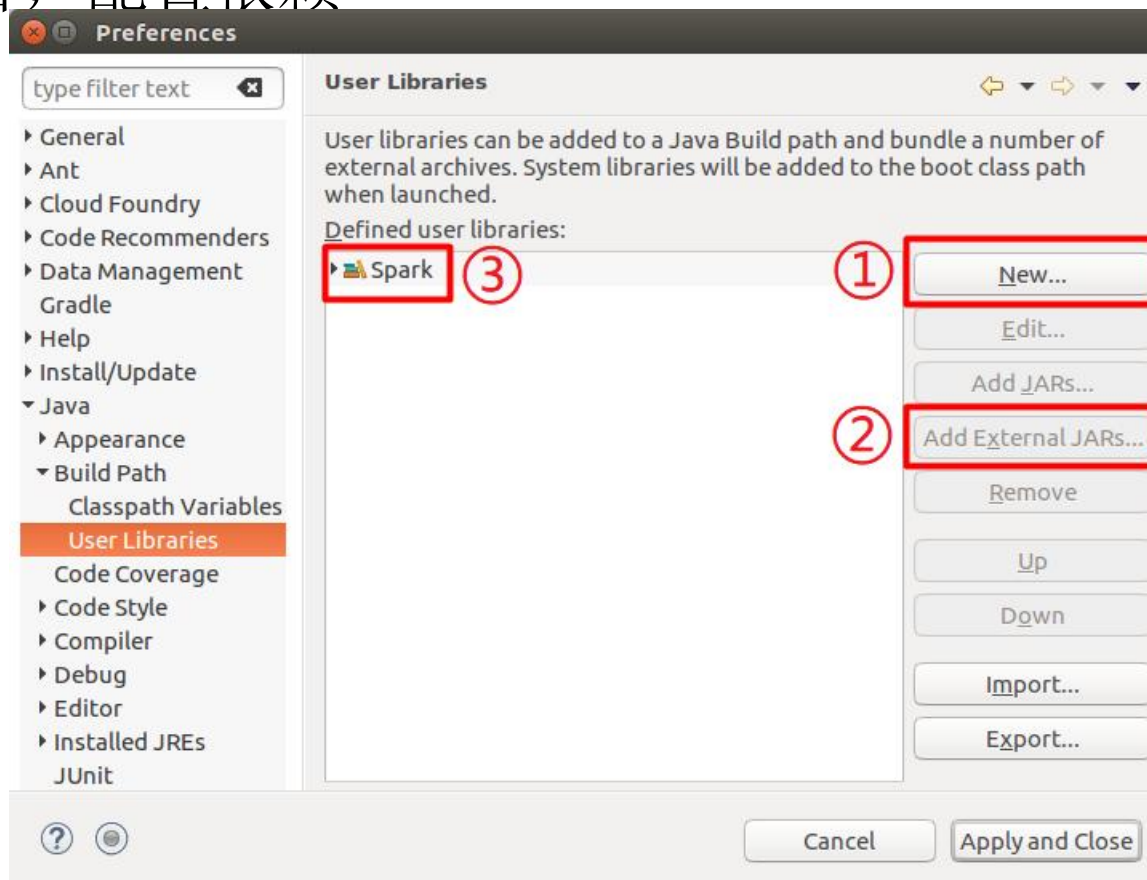
2.4.2 Eclipse的实例（Scala）

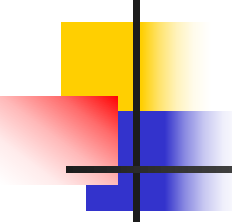
2.新建项目，配置依赖

（5）在路径选择窗口中，选择Spark安装目录下的jars文件夹，添加jars文件下的全部jar包。

2.4.2 Eclipse的实例（Scala）

2.新建项目，配置依赖



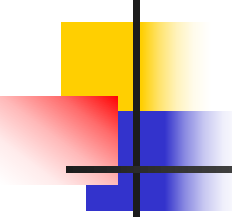


2.4.2 Eclipse的实例（Scala）

2.新建项目，配置依赖

注意：为了方便，也可以将Spark的所有jar包作为User Library添加到Eclipse中，步骤为：

- 1) 点击Eclipse菜单栏中的【Window】，选择弹出选项中的【Preference】，弹出如图所示窗口；
- 2) 依次展开窗口左侧的【Java】>>【Build Path】>>【User Libraries】，并选中【User Libraries】；
- 3) 点击右侧的【New】，输入名称，本书使用Spark为用户Library命名，如上图编号①所示；
- 4) 点击【Add External JARs】，如编号②所示，定位到Spark安装目录下的jar文件夹，将Spark安装目录下的所有jar包添加进来，并点击【OK】完成导入，此时窗口中会出现定义好的User Library，如编号③所示。



2.4.2 Eclipse的实例（Scala）

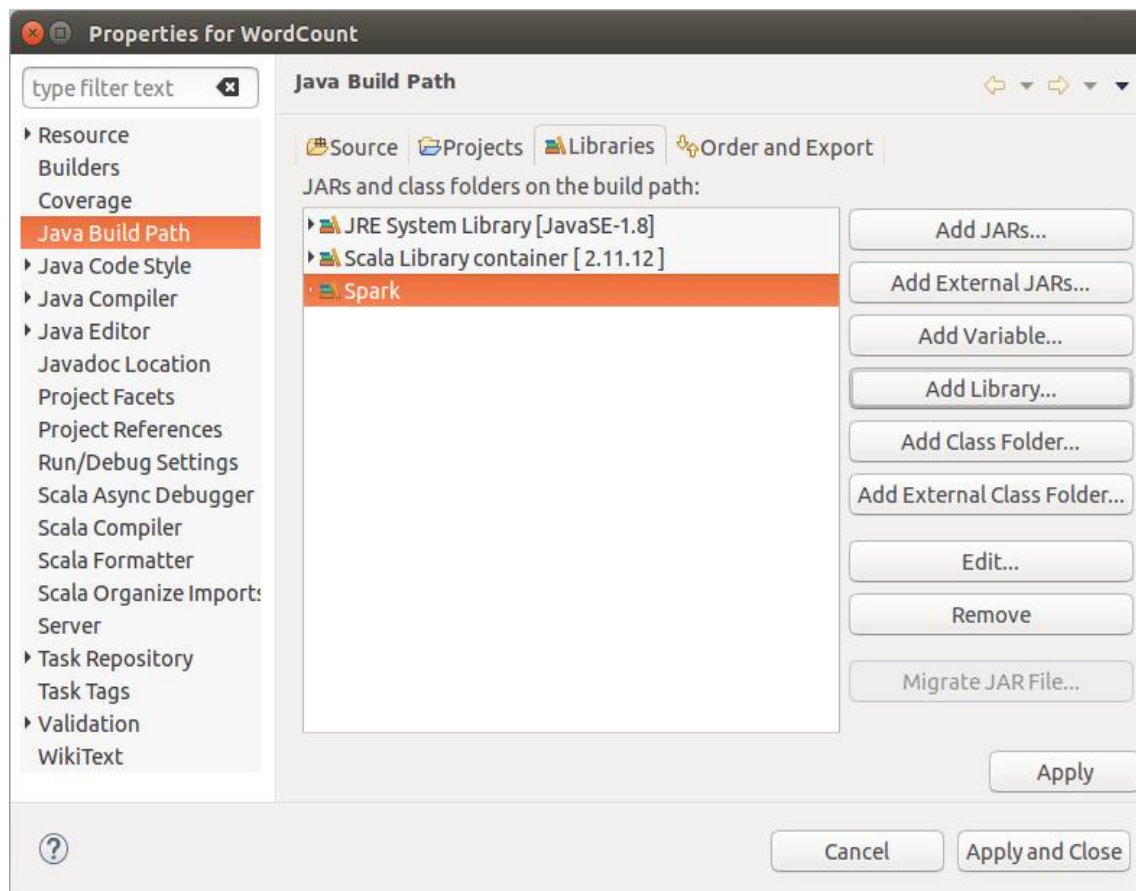
2.新建项目，配置依赖

Spark的jar包添加至User Library以后，下次需要导入Spark的jar包时，仅需将该User Library添加至项目的Java Build Path中。以WordCount项目为例，步骤为：

- 1) 右键点击项目名【WordCount】，再点击弹出选项中的【Properties】，出现如图所示窗口；
- 2) 点击左侧框中的【Java Build Path】，再点击右侧【Add Library】，选择【User Libraries】，在中间窗口中选中已经添加好的User Library（Spark），并点击【OK】完成添加。

2.4.2 Eclipse的实例（Scala）

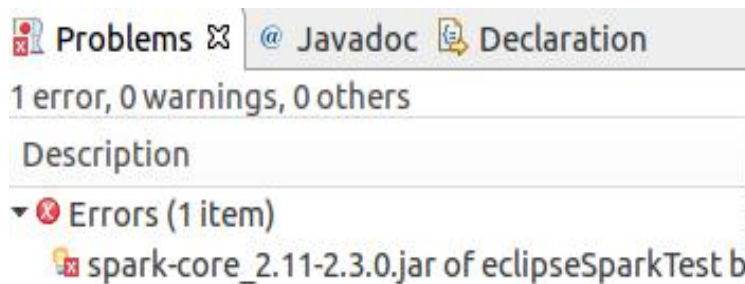
2.新建项目，配置依赖



2.4.2 Eclipse的实例（Scala）

2.新建项目，配置依赖

（6）点击【Apply and Close】，完成项目的创建，此时出现如图所示的错误。



该错误主要是当前安装的Spark对应的Scala版本号与Eclipse中的Scala插件对应的版本号不一致导致的。因此，修改Scala插件对应的版本。

2.4.2 Eclipse的实例（Scala）

2.新建项目，配置依赖

解决错误，具体步骤为：

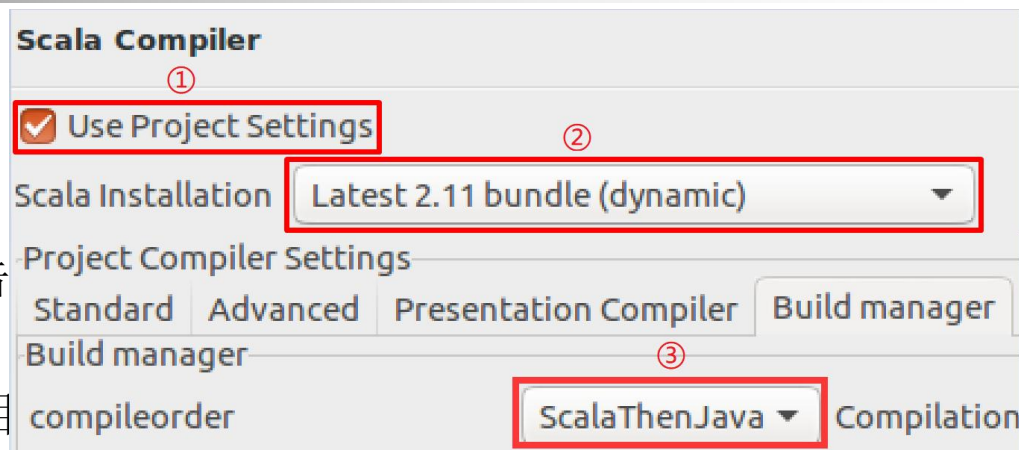
1) 在项目名“eclipseSparkTest”上点击鼠标右键，点击弹出选项中的

【Properties】，在弹出窗口中修改项目的依赖。选中左侧框中的【Scala Compiler】，右侧面板会出现如图所示信息：

2) 勾选【Use Project Settings】，参考图中编号①，对【Scala Installation】进行修改。本书中安装的Spark对应的Scala依赖包版本是2.11.8，选择【Latest 2.11 bundle(dynamic)】，如图中编号②。

3) 将【Build manager】区域中的【compileorder】设置为【ScalaThenJava】，如图中编号③。

4) 点击右下角的【Apply and Close】，完成Scala依赖包版本的修改。项目的环境错误提示信息消失，说明环境配置正确。

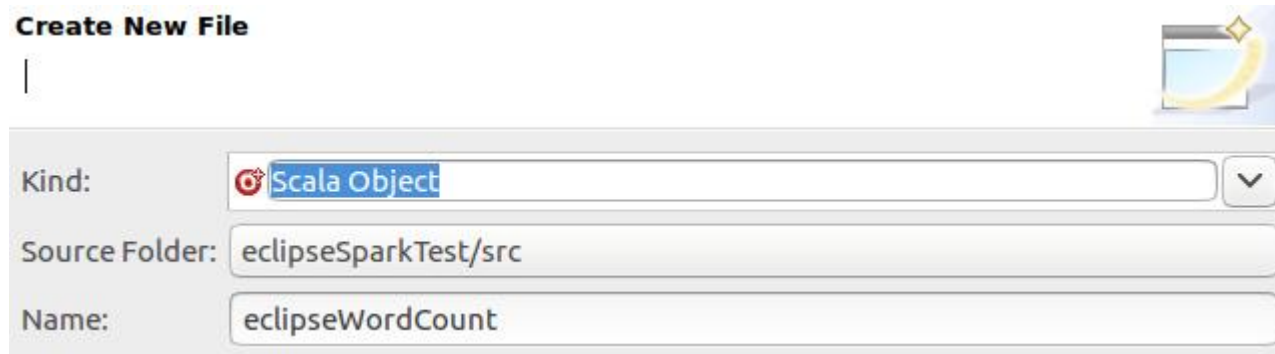


2.4.2 Eclipse的实例（Scala）

3.编写程序

（1）右键选中eclipseSparkTest/src文件夹，创建package。方法为：依次点击【New】>>【Package】，设置包名称（本书实例中，包名为eclipseSparkTest），最后点击【Finish】。

（2）右键包文件夹创建Scala文件。依次点击【New】>>【Others】>>【Scala Wizards】>>【Scala Object】。在图示Scala类的设置页面中，将类命名为eclipseWordCount。最后点击【Finish】，完成Scala类的创建。



（3）在eclipseWordCount中输入代码。

2.4.2 Eclipse的实例（Scala）

4.编译与运行

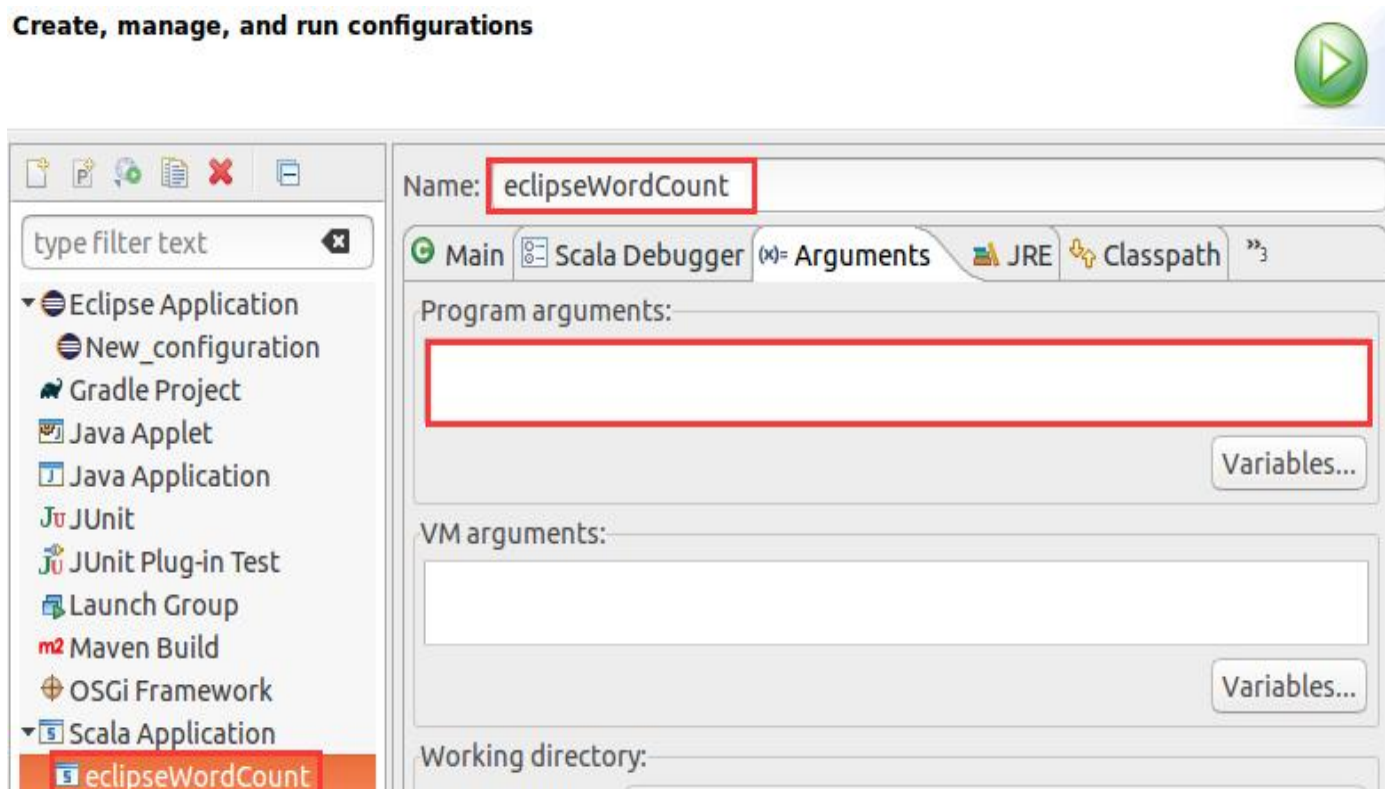
（1）点击菜单栏快捷选项框的运行按钮，如图所示。完成编译。

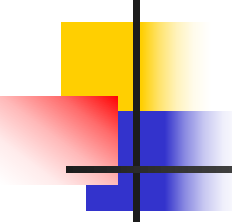


（2）点击如图所示的运行按钮右侧的箭头。在下拉框中，选择【Run Configuration s...】。在弹出的配置界面设置运行程序的名称和参数。双击左侧选项框中的【Scala Application】，右侧面板会出现应用程序名称和参数的设置界面。如图所示：

2.4.2 Eclipse的实例（Scala）

4.编译与运行



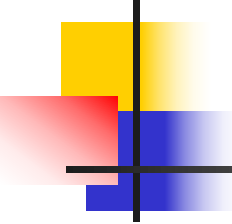


2.4.2 Eclipse的实例（Scala）

4.编译与运行

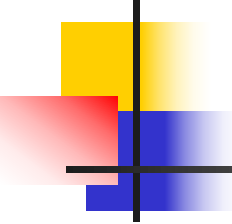
（3）如上图所示，在【Name】框中，设置运行程序的名称。在【Program arguments】框中，设置参数。实例代码args(0)表示输入文件路径，填写“file:///usr/local/sparkwc”。设置完成以后，点击【Run】运行程序，在控制台出现运行结果：

(scala,1) (spark,3) (hadoop,1) (R,1) (python,1) (hello,2) (java,1)



2.5 本章小结

本章主要介绍了Linux环境中以Local模式部署Spark的流程；IDEA创建并运行Spark应用程序的流程；IDEA进行应用程序打包、spark-submit提交运行jar包的流程、Eclipse创建并运行Spark应用程序的流程。依据本章完成Spark实战开发环境搭建。使用该环境，快速开发Spark应用程序。

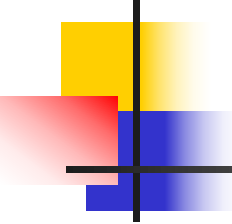


思考与习题

1.在Spark安装目录下有一些示例程序，包括Scala、Python、R、Java语言的版本。以本书安装的Spark环境为例，Scala语言的示例程序文件夹位于“`/usr/local/spark-2.3.0-bin-hadoop2.7/examples/src/main/scala/org/apache/spark/examples`”。在这个文件夹下有一个名为SparkPi.scala的程序，功能是计算 π 的近似值。Spark提供了一个shell脚本程序专门用于运行示例，在spark安装根目录的bin文件夹下，名为run-example。请读者用run-example运行SparkPi程序，并用IDEA或Eclipse查看该程序的源码。

运行方法：

- (1) 打开一个终端，输入命令：`cd /usr/local/spark-2.3.0-bin-hadoop2.7/bin`
- (2) 继续输入命令：`./run-example SparkPi`
- (3) 在运行结果中找到有“Pi is roughly”字样的一行即为结果。



思考与习题

2. 请简述推荐用Linux系统搭建Spark开发环境的原因。
3. 请说明在spark-shell中编程与在IDEA中编程有什么区别。
4. 请简述SparkWEB可以查看哪些信息。
5. 请参考2.2.3小节，说明为什么在sparkwc还未建立的情况下，

```
val rdd1 = sc.textFile("file:///usr/local/sparkwc")
```

不会报错。
6. 请说明Eclipse与IDEA的主要区别。



Spark大数据 编程基础 (Scala版)

